

VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY

- UNIVERSITY OF INFORMATION TECHNOLOGY -



PROJECT REPORT

**JANUS SCALE:
AN EVENT-DRIVEN KUBERNETES AUTOSCALING &
SELF-HEALING FRAMEWORK**

Instructor: MSc. Do Hoang Hien

Student: Dao Minh Chau - 24560025

HO CHI MINH CITY

September 21, 2026

Table of Contents

Table of Contents	2
List of Figures	4
List of Tables	5
List of Abbreviations	6
1. Introduction.....	7
1.1 Background.....	7
1.2 Problem Statement.....	7
1.3 Motivation.....	8
1.4 Objectives.....	9
1.5 Scope.....	9
1.6 Contributions.....	10
1.7 Report Organization.....	11
2. Background and Related Technologies.....	12
2.1 Kubernetes Autoscaling and Existing Autoscaling Approaches.....	12
2.2 Telemetry and Observability.....	13
2.3 MEWMA-based Anomaly Detection.....	14
2.4 FFT-Based Temporal Analysis.....	15
2.5 GARCH/DCC-Based Dependency Analysis.....	16
2.6 Policy-based Actuation.....	17
2.7 Related Autoscaling Approaches.....	17
3. System Analysis and Requirements.....	19
3.1 System Context.....	19
3.2 Functional Requirements.....	19
3.3 Non-functional Requirements.....	21
3.4 Constraints and Assumptions.....	23
3.5 Design Goals.....	24
4. System Design and Architecture.....	25
4.1 Overall Architecture.....	25
4.2 Telemetry-to-Decision Pipeline.....	26
4.3 Decision and Enforcement Flow.....	28
4.4 Calibration and Evaluation Architecture.....	29
4.5 Data Flow.....	30
4.6 Control Flow.....	32
4.7 Safety and Operational Controls.....	32
5. Implementation.....	33
5.1 P1 - Ingestion.....	33
5.2 P2 - Preprocessing.....	35
5.3 P3 - Detection.....	37
5.4 P4 - Attribution.....	39

5.5 P5 - Decision.....	40
5.6 P6 - Enforcement.....	41
5.7 P7 - Calibration and Evaluation.....	43
5.8 P8 - Operations and Safety.....	44
5.9 Configuration and Runtime Parameters.....	45
5.10 Kubernetes / Actuator Integration.....	47
6. Experiment and Evaluation.....	49
6.1 Experimental Setup.....	49
6.2 Dataset / Runtime Trace.....	50
6.3 Experimental Scenarios.....	52
6.4 Detection Evaluation.....	53
6.5 Attribution Evaluation.....	54
6.6 Decision / Scaling Evaluation.....	55
6.7 Actuation Verification.....	57
6.8 Calibration and Candidate Evaluation.....	58
6.9 Limitations of the Evaluation.....	63
7. Results and Discussion.....	65
7.1 Detection Results.....	65
7.2 Attribution Results.....	66
7.3 Closed-loop Scaling Evidence.....	67
7.4 Candidate / Shadow Evaluation.....	68
7.5 Interpretation of Results.....	69
7.6 Limitations and Threats to Validity.....	70
7.7 Comparison Between Intended Design and Actual Implementation.....	71
8. Conclusion and Future Work.....	74
8.1 Summary.....	74
8.2 Achieved Objectives.....	74
8.3 Current Limitations.....	77
8.4 Future Improvements.....	77
Appendices.....	79
A. Evidence Checklist.....	79
B. Figures Checklist.....	79
C. Tables Checklist.....	80
D. Claims That Must Not Be Overstated.....	80
E. Missing Evidence / Open Questions.....	81
References.....	82

List of Figures

Figure 4.1: JanusScale Overall System Architecture.....26

Figure 4.2: Telemetry-to-decision Data Flow..... 27

Figure 4.3: Decision and Recovery/Enforcement Flow.....29

Figure 4.4: Calibration & Evaluation Architecture.....30

List of Tables

Table 2.1: Comparison of Autoscaling Approaches.....	18
Table 3.1: Functional Requirements Traced to Logical Planes.....	20
Table 3.2: Non-Functional Requirements.....	21
Table 4.1: Data Mapping by Logical Plane.....	31
Table 5.1: run.sh Configuration Parameters.....	45
Table 6.1: Experimental Runtime Parameters (run.sh Values Used in the Evaluation).....	49
Table 6.2: Timeline of Actuator Frontend Replica-Change Events.....	51
Table 6.3: Detection Evidence Summary (P3 Detection Record Fields).....	54
Table 6.4: Attribution Evidence Summary (P4 Attribution Record Fields).....	55
Table 6.5: Interpretation of P5 Decision Record Fields.....	56
Table 6.6: Field-Level Comparison Between the JanusScale Log and the Actuator Log.....	58
Table 6.7: Shadow Candidate Configurations (eval.json).....	59
Table 6.8: Summary of out_*.txt Evaluation Results.....	59
Table 6.9: False-Alarm Rate (FAR) Grid Evaluation Results (out_far.txt).....	61
Table 6.10: Latch and Cooldown Evaluation Results (out_latch.txt).....	61
Table 7.1: Univariate/Multivariate Detection Agreement Summary (out_uni2.txt).....	65
Table 7.2: Attribution Result Field Summary.....	66
Table 7.3: Candidate/Shadow Evaluation Configurations.....	68
Table 7.4: Proposal vs. Actual Implementation Status.....	71
Table 8.1: Objectives vs. Supporting Evidence.....	74

List of Abbreviations

Established technical terms are defined directly below. Project-specific log-line prefixes (UNI, D, ATTR, REP, DECIDE, ACT) are used consistently in the runtime evidence quoted in handoff.md, but their authoritative field-by-field definitions should still be confirmed against source-code comments before being asserted in the final report.

Term	Meaning / Note
P1–P8	JanusScale logical planes: Ingestion, Preprocessing, Detection, Attribution, Decision, Enforcement, Calibration & Evaluation, Operations & Safety
MEWMA	Multivariate Exponentially Weighted Moving Average
T^2	Hotelling's T-squared statistic (multivariate anomaly statistic used by P3 Detection)
UCL	Upper Control Limit
FAR	False Alarm Rate
FFT	Fast Fourier Transform
GARCH	Generalized Autoregressive Conditional Heteroskedasticity
DCC	Dynamic Conditional Correlation
VE	Variance-explained gating step used in <code>fft_fit</code> / <code>fft_slow</code> - exact definition to be confirmed from source comments
RPS	Requests Per Second
CPU / RAM	Compute / memory resource dimensions of the 6-D observation vector $y[t]$
ASYM	Asymmetry-related signal dimension in $y[t]$
SLOW	Slow-request / latency-related signal dimension in $y[t]$
HMAC-SHA256	Hash-based Message Authentication Code using SHA-256, used to sign actuation requests
K8s	Kubernetes
API	Application Programming Interface
JSONL	JSON Lines (newline-delimited JSON), used for actuator audit logging
GitOps	Git-based operations/deployment workflow (proposal-level concept; see Section 7.7)
OTel	OpenTelemetry (proposal-level concept; see Section 7.7)

1. Introduction

1.1 Background

Modern microservice applications are commonly deployed on Kubernetes, where multiple services operate concurrently and their resource requirements can vary with workload conditions. Maintaining appropriate resource capacity therefore requires continuous observation of runtime behavior and mechanisms that can adapt workload resources to changing conditions.

Conventional autoscaling approaches commonly rely on individual resource or application metrics. However, runtime behavior in a distributed application can involve changes across multiple correlated telemetry dimensions. A scaling mechanism can therefore benefit from analyzing these observations jointly and identifying abnormal changes before selecting an appropriate scaling action.

JanusScale is developed in this context as an **event-driven Kubernetes autoscaling and self-healing framework**. Its central anomaly-detection mechanism is based on **Multivariate Exponentially Weighted Moving Average (MEWMA)** analysis and the associated multivariate T^2 statistic. Runtime telemetry is represented as a multivariate observation so that changes across multiple signals can be analyzed together rather than treated as isolated threshold violations.

In addition to MEWMA-based detection, JanusScale incorporates **FFT-based temporal analysis** and **GARCH/DCC-related dependency analysis** as supporting mechanisms within the processing pipeline. FFT is used to extract temporal characteristics from selected runtime signals, while the GARCH/DCC-related components provide additional information about signal variability and dependencies. These mechanisms complement the central MEWMA-based detection process and provide information for subsequent attribution and scaling decisions.

The online processing pipeline is organized into six logical planes: **Ingestion (P1)**, **Preprocessing (P2)**, **Detection (P3)**, **Attribution (P4)**, **Decision (P5)**, and **Enforcement (P6)**. Runtime telemetry is collected and transformed into a multivariate representation, analyzed for anomalous behavior, associated with relevant dimensions and workload dependencies, and subsequently used to determine and enforce a scaling action.

The project therefore focuses on establishing an **end-to-end closed-loop autoscaling workflow** in Kubernetes, from runtime observation and multivariate analysis to workload enforcement. The evaluation emphasizes the ability of the implemented pipeline to complete this closed loop and the runtime overhead introduced by the analysis and decision process.

1.2 Problem Statement

In a Kubernetes-based microservice system, workload conditions can change over time, resulting in variations in resource utilization and service behavior. A scaling mechanism based only on individual resource thresholds may not adequately capture abnormal conditions involving multiple telemetry dimensions or determine which workload is associated with the observed change.

JanusScale addresses this problem as an **event-driven, telemetry-driven autoscaling problem**. Runtime telemetry must first be transformed into a multivariate representation of the system state. The resulting observations are then analyzed to detect abnormal behavior using **MEWMA and the associated T^2 statistic**, with **FFT-based temporal analysis** and **GARCH/DCC-related dependency information** providing supporting information for the analysis.

Detection alone, however, does not directly determine which workload should be scaled. Therefore, the detected behavior must be **attributed to relevant signal dimensions and workload dependencies** before a scaling decision can be made. The decision must then be translated into a controlled actuation request that changes the replica count of the target Kubernetes workload.

Accordingly, the problem addressed by JanusScale is the design and implementation of an online pipeline that connects **multivariate anomaly detection, temporal and dependency analysis, anomaly attribution, scaling decision, and workload enforcement** into a closed-loop autoscaling workflow.

The project focuses on establishing and evaluating this technical closed loop rather than proving an optimal autoscaling policy. In particular, the evaluation considers whether the implemented pipeline can propagate an observed runtime condition through detection, attribution, decision, and enforcement while keeping the additional runtime overhead of the analysis pipeline within a practical range.

1.3 Motivation

The motivation for JanusScale is to develop an autoscaling pipeline that can reason about **multivariate changes in runtime behavior** rather than relying solely on isolated resource thresholds. In a distributed Kubernetes workload, resource utilization and service behavior may change across multiple telemetry dimensions. MEWMA-based multivariate analysis provides the central mechanism for detecting such changes while preserving the relationship between the observed dimensions.

A second motivation is to complement multivariate detection with **temporal and dependency information**. FFT-based analysis provides supporting information about periodic characteristics in selected runtime signals, while GARCH/DCC-related analysis provides additional information about conditional variability and dependencies between signals. These mechanisms are incorporated as supporting components of the analysis pipeline rather than as replacements for the central MEWMA-based detection mechanism.

A third motivation is to connect anomaly detection to an actionable **scaling decision**. Detecting an anomalous observation alone does not identify which workload should be scaled. JanusScale therefore separates detection from attribution, allowing information about the anomalous dimension and workload dependency to be incorporated into the decision process.

Finally, the project is motivated by the need to demonstrate that these analysis components can operate as part of a **complete online closed loop**. The system connects telemetry processing, multivariate analysis, attribution, decision, and enforcement so that a detected

runtime condition can ultimately result in a controlled change to the Kubernetes workload. The evaluation therefore focuses on demonstrating this closed-loop behavior and characterizing the runtime overhead introduced by the implemented pipeline.

1.4 Objectives

The objectives of the JanusScale project are to develop and evaluate an event-driven Kubernetes autoscaling pipeline that integrates multivariate anomaly detection, supporting signal analysis, attribution, decision-making, and controlled workload enforcement. Specifically, the project aims to:

- **Implement an end-to-end online autoscaling pipeline** covering telemetry ingestion, preprocessing, detection, attribution, decision, and enforcement through the logical planes P1–P6.
- **Implement MEWMA-based multivariate anomaly detection** using the associated T^2 statistic as the central mechanism for identifying abnormal runtime behavior.
- **Incorporate supporting temporal and dependency analysis** through FFT-based signal analysis and GARCH/DCC-related analysis within the runtime processing pipeline.
- **Integrate anomaly attribution with scaling decisions** so that detected anomalous behavior can be associated with relevant signal dimensions and workload dependencies before a scaling action is selected.
- **Demonstrate a complete closed-loop scaling workflow** in which a detected runtime condition can propagate through the decision and enforcement stages to produce a controlled change in Kubernetes workload capacity.
- **Evaluate the runtime overhead of the implemented pipeline**, with emphasis on the processing cost introduced by telemetry analysis and the mechanisms required to complete the online scaling loop.

The project does not aim to establish an optimal autoscaling policy or a universally optimal system configuration.

1.5 Scope

This project focuses on the implementation and evaluation of the **online JanusScale autoscaling loop** in a Kubernetes environment. The primary scope is to establish a complete path from runtime telemetry to workload enforcement and to characterize the runtime overhead introduced by the implemented processing and decision pipeline.

The scope covers the following logical planes:

- **Telemetry Ingestion (P1):** collecting runtime telemetry from monitored Kubernetes workloads.
- **Preprocessing (P2):** transforming telemetry into the multivariate observations required by the analysis pipeline and generating supporting signal features.
- **Detection (P3):** applying **MEWMA and the associated T^2 statistic** as the central multivariate anomaly-detection mechanism, together with supporting **FFT-based temporal analysis**.

- **Attribution (P4):** identifying relevant signal dimensions and workload dependencies associated with detected anomalous behavior, with **GARCH/DCC-related analysis** providing supporting dependency information.
- **Decision (P5):** determining whether a scaling action should be taken and selecting the corresponding replica target under the implemented runtime constraints.
- **Enforcement (P6):** transmitting the scaling request to the actuator and applying the resulting replica change to the Kubernetes workload.

A primary focus of the project is to demonstrate a **complete closed-loop scaling workflow**, in which an observed runtime condition propagates through analysis, attribution, decision, and enforcement, with the resulting workload state observable through subsequent telemetry.

The evaluation also focuses on the **runtime overhead** of the implemented pipeline, including the processing cost associated with multivariate detection and the supporting FFT and GARCH/DCC-related components. The purpose is to characterize the operational cost of the implemented closed loop rather than to establish an optimal autoscaling policy.

Offline calibration, candidate selection, and shadow evaluation are not treated as primary objectives of the project. Where available, they are considered supporting experimental procedures rather than part of the main closed-loop scope. The project therefore does not claim to establish optimal autoscaling performance or a universally optimal system configuration.

1.6 Contributions

The main contributions of the JanusScale project are:

- **An end-to-end event-driven Kubernetes autoscaling framework** that connects telemetry ingestion, preprocessing, multivariate analysis, attribution, scaling decision, and workload enforcement into a single online processing pipeline.
- **A MEWMA-based multivariate anomaly-detection pipeline** using the T^2 statistic as the central detection mechanism for identifying abnormal runtime behavior across multiple telemetry dimensions.
- **Integration of complementary signal-analysis mechanisms**, including FFT-based temporal analysis and GARCH/DCC-related dependency analysis, to provide additional information for the detection and attribution stages.
- **A dependency-aware attribution and decision path** that connects detected anomalous behavior to a relevant signal dimension and workload before determining the scaling action.
- **A controlled Kubernetes enforcement mechanism** that translates scaling decisions into replica changes through a dedicated actuation path, enabling the system to complete an online closed-loop scaling workflow.
- **An empirical evaluation of the implemented pipeline**, focusing on demonstrated closed-loop scaling behavior and the runtime overhead introduced by the analysis and decision components.

These contributions are limited to the capabilities demonstrated by the implemented system and available experimental evidence; the project does not claim optimal autoscaling performance or a universally optimal configuration.

1.7 Report Organization

The remainder of this report is organized as follows.

Chapter 2, Background and Related Technologies, presents the theoretical and technical foundations of JanusScale, including Kubernetes autoscaling, telemetry and observability, MEWMA-based multivariate anomaly detection, FFT-based temporal analysis, GARCH/DCC-related dependency analysis, and policy-controlled workload actuation.

Chapter 3, System Analysis and Requirements, defines the system requirements, functional and non-functional requirements, operational constraints, and the expected behavior of the JanusScale autoscaling pipeline.

Chapter 4, System Design and Architecture, describes the overall architecture and data flow of JanusScale. It presents the logical planes P1–P8, the online telemetry-to-decision pipeline, the closed-loop control flow, and the supporting calibration and evaluation structure.

Chapter 5, Implementation, details the implementation of the JanusScale components, including telemetry ingestion, preprocessing, MEWMA-based detection, FFT processing, attribution, GARCH/DCC-related analysis, decision logic, safety controls, and Kubernetes workload enforcement.

Chapter 6, Experiment and Evaluation, describes the experimental environment and evaluates the implemented system through runtime traces, closed-loop scaling events, detection and attribution results, and measurements of the runtime overhead introduced by the pipeline.

Chapter 7, Results and Discussion, discusses the experimental findings, including the demonstrated closed-loop behavior, observed system overhead, and limitations of the available evaluation evidence. The chapter also distinguishes implemented capabilities from aspects that remain outside the demonstrated scope.

Chapter 8, Conclusion and Future Work, summarizes the implemented system and experimental findings, and identifies potential directions for extending and improving JanusScale beyond the current project scope.

2. Background and Related Technologies

2.1 Kubernetes Autoscaling and Existing Autoscaling Approaches

Kubernetes provides native mechanisms for automatically adjusting the capacity of workloads. The **Horizontal Pod Autoscaler (HPA)** automatically adjusts the replica count of a scalable workload according to configured metrics. The HPA controller periodically evaluates observed metrics and updates the desired scale of the target workload. Supported inputs include resource metrics such as CPU and memory as well as custom metrics.

HPA represents a **reactive metric-driven autoscaling** approach: the scaling decision is derived from the currently observed system metrics. This provides a general mechanism for horizontal scaling, but the autoscaling controller itself does not perform the multivariate anomaly analysis used by JanusScale. In JanusScale, telemetry is processed through a dedicated analysis pipeline before a scaling decision is generated.

For workloads whose scaling conditions are driven by external events, **Kubernetes Event-driven Autoscaling (KEDA)** provides an additional approach. KEDA operates alongside Kubernetes HPA and exposes metrics derived from external event sources to the Kubernetes autoscaling mechanism. Its scalers can monitor sources such as message queues, databases, APIs, and Prometheus metrics. KEDA also supports scaling from zero to one directly through the KEDA operator, while one-to-N and N-to-one scaling for Deployments and StatefulSets is handled through HPA.

Another direction explored in the literature is **predictive autoscaling based on workload forecasting**. Instead of reacting only to the current observed state, predictive approaches use historical workload or resource-utilization data to estimate future demand and use the prediction to support resource-allocation decisions.

Dang-Quang and Yoo proposed a **deep learning-based autoscaling approach using Bidirectional Long Short-Term Memory (Bi-LSTM) for Kubernetes**, illustrating the use of recurrent neural-network-based time-series prediction in Kubernetes autoscaling. Yuan and Liao presented a **time-series-based approach to elastic Kubernetes scaling**, further investigating the use of historical workload behavior for elastic resource management in Kubernetes. In a broader edge-cloud setting, Liu et al. studied **workload forecasting-based elastic resource management**, using workload forecasting as a basis for elastic resource management under changing demand.

These approaches represent different ways of deriving scaling signals. **HPA** primarily uses observed resource or custom metrics, while **KEDA** extends event-driven scaling through external event sources. Forecasting-based approaches such as Bi-LSTM and other time-series methods attempt to estimate future workload conditions before determining the required capacity.

JanusScale follows a different design point: **event-driven autoscaling based on multivariate anomaly analysis**. Its central detection mechanism is MEWMA with the associated T^2 statistic. FFT-based temporal analysis and GARCH/DCC-related analysis are incorporated as supporting mechanisms, after which anomaly attribution and decision logic determine the target scaling action. The resulting decision is then enforced through the

Kubernetes workload scaling interface. Thus, JanusScale is positioned as an anomaly-aware approach that complements, rather than replaces, the reactive, event-driven, and forecasting-based approaches described above.

2.2 Telemetry and Observability

A telemetry-driven autoscaling system requires runtime information that reflects both the resource state of workloads and their service behavior. In a Kubernetes-based microservice environment, relevant observations can include resource utilization, request rate, error behavior, and latency-related measurements. These signals provide different views of the current workload condition and can therefore be combined into a multivariate representation for subsequent analysis.

JanusScale collects runtime telemetry from the monitored Kubernetes environment and processes it before passing the resulting observations to the anomaly-detection pipeline. The implementation uses telemetry related to **CPU utilization, memory utilization, request rate (RPS), error behavior, asymmetry-related behavior (ASYM), and slow-request behavior (SLOW)**. These signals are combined into a six-dimensional observation:

$$y_t = [t_{cpu}, r_d, RPS, \text{logit}(err_rate), \log(ASYM), \text{logit}(SLOW)]$$

where each component represents a different aspect of the observed runtime state.

The resulting six-dimensional observation is used as the input to the multivariate processing pipeline. The system maintains a rolling observation window for subsequent analysis, allowing the detection mechanism to consider recent system behavior rather than only the latest individual measurement.

The telemetry pipeline also provides information required by the subsequent attribution and decision stages. In particular, workload-level information allows detected anomalous behavior to be associated with a target dependency or workload, while request-rate and resource-related measurements provide context for determining the appropriate scaling action.

Within JanusScale, telemetry therefore serves two purposes. First, it provides the **multivariate input for anomaly detection**, with MEWMA and T^2 forming the central detection mechanism. Second, it provides the **runtime context required for attribution and scaling decisions**. This design allows the subsequent stages of the pipeline to operate on a common representation of the observed system state rather than relying on a single isolated metric.

2.3 MEWMA-based Anomaly Detection

JanusScale uses **Multivariate Exponentially Weighted Moving Average (MEWMA)** as the central mechanism for detecting abnormal runtime behavior. MEWMA is suitable for monitoring systems in which multiple observed variables may change simultaneously and where the relationship between these variables is relevant to the overall system state.

Let the runtime observation at time t be represented by the six-dimensional vector

$$y_t = [t_{cpu}, r_d, RPS, \text{logit}(\text{err_rate}), \text{log}(\text{ASYM}), \text{logit}(\text{SLOW})].$$

Given a baseline mean vector μ_0 , the MEWMA statistic is updated recursively as

$$z_t = \lambda(y_t - \mu_0) + (1 - \lambda)z_{t-1},$$

where $0 < \lambda \leq 1$ is the smoothing parameter. The recursive formulation gives greater influence to recent observations while retaining information from previous observations.

The resulting MEWMA state is evaluated using a multivariate quadratic statistic. In general form, the statistic can be expressed as

$$T_t^2 = z_t^T \Sigma_z^{-1} z_t,$$

where Σ_z represents the covariance structure of the MEWMA statistic. The resulting T^2 value is compared with an upper control limit (UCL). An observation is considered anomalous when the statistic exceeds the applicable detection threshold.

This multivariate formulation allows JanusScale to consider deviations across multiple telemetry dimensions jointly. For example, an abnormal condition may involve changes in resource utilization together with request or service-behavior signals. The resulting T^2 statistic provides a single multivariate measure that can be passed to the subsequent attribution stage.

In the JanusScale pipeline, MEWMA detection is performed over a rolling observation window rather than treating each telemetry sample as an independent event. The implementation uses a production window of **W=240W=240 observations**. The value represents the number of observations maintained by the analysis window and should not be interpreted directly as 240 seconds.

MEWMA therefore forms the central detection stage between telemetry preprocessing and anomaly attribution:

Telemetry $\rightarrow$ Multivariate Observation $\rightarrow$ MEWMA/ T^2 $\rightarrow$ Anomaly $\rightarrow$ Attribution.

Within JanusScale, the MEWMA-based detector is complemented by FFT-based temporal analysis and GARCH/DCC-related analysis. These mechanisms provide additional information to the processing pipeline, while MEWMA/ T^2 remains the primary multivariate anomaly-detection mechanism.

2.4 FFT-Based Temporal Analysis

Runtime behavior in a Kubernetes workload can contain temporal patterns that are not directly captured by instantaneous resource measurements. JanusScale therefore incorporates **Fast Fourier Transform (FFT)** analysis as a supporting mechanism for identifying periodic characteristics in selected runtime signals.

For a discrete signal $x[n]$ containing N observations, the discrete Fourier transform can be expressed as

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N},$$

where $X[k]$ represents the contribution of the corresponding frequency component. The magnitude

$$|X[k]|$$

can then be used to characterize the strength of individual frequency components in the observed signal.

In JanusScale, FFT analysis is applied to selected signals rather than to the complete six-dimensional observation vector. The implementation uses FFT-derived information for **CPU, RPS, ASYM, and SLOW**, while **RAM and error-rate signals remain in their direct representations** within the processing pipeline.

Before extracting useful frequency components, the implementation applies preprocessing and validation steps to reduce the influence of unsuitable spectral components. These include trend handling, robust statistics based on the median and median absolute deviation (MAD), peak selection and grouping, and validity gates applied to the resulting spectral representation. The implementation also constrains the number of selected frequency components and applies variance-explained (VE) criteria when determining whether the extracted spectral information is sufficiently representative.

The resulting FFT features provide information about **temporal periodicity and recurring behavior** in selected runtime signals. This information is used as a supporting component of the JanusScale analysis pipeline rather than as the primary anomaly detector. The central multivariate detection mechanism remains MEWMA and the associated T^2 statistic.

Conceptually, the role of FFT in the pipeline can be represented as:

Runtime Signal $\rightarrow$ *Preprocessing* $\rightarrow$ *FFT* $\rightarrow$ *Spectral Features* $\rightarrow$ *Supporting Analysis*

Thus, FFT provides a temporal/spectral view of selected runtime signals that complements the multivariate statistical view provided by MEWMA.

2.5 GARCH/DCC-Based Dependency Analysis

Runtime behavior in a distributed system may exhibit changes in both variability and the relationships between observed signals. JanusScale therefore incorporates **GARCH/DCC-related analysis** as a supporting component of its runtime analysis pipeline.

The **GARCH** component provides a way to represent time-varying conditional variance. A simplified GARCH formulation is

$$\sigma_t^2 = \omega + \alpha \epsilon_{t-1}^2 + \beta \sigma_{t-1}^2,$$

where σ_t^2 represents the conditional variance, ϵ_t is the model residual, and ω , α , and β are model parameters.

The **Dynamic Conditional Correlation (DCC)** component extends this idea to the relationships between multiple signals. A simplified representation of its time-varying dependency structure is

$$Q_t = (1 - a - b) \bar{Q} + a \epsilon_{t-1} \epsilon_{t-1}^T + b Q_{t-1},$$

where $\bar{Q}$ represents the baseline correlation structure and Q_t represents its time-varying counterpart.

In JanusScale, these mechanisms provide **supporting information about conditional variability and dependencies between runtime signals**. They complement the central MEWMA/ T^2 detection mechanism rather than replacing it. While MEWMA is responsible for identifying multivariate deviations, GARCH/DCC-related information can provide additional context when interpreting the observed system state and its dependencies.

Therefore, the role of GARCH/DCC in JanusScale is primarily to enrich the analysis used by subsequent attribution and decision stages. It is not treated as an independent autoscaling mechanism or as the primary anomaly detector.

Conceptually, the three analytical components have complementary roles:

$$MEWMA/T^2 \rightarrow \text{Multivariate Anomaly}$$

$$FFT \rightarrow \text{Temporal Characteristics}$$

$$GARCH/DCC \rightarrow \text{Variability and Dependency Information}$$

Together, these components provide different views of runtime behavior before the system proceeds to **attribution and scaling decisions**.

2.6 Policy-based Actuation

After the detection, attribution, and decision stages determine that a scaling action should be performed, JanusScale requires a controlled mechanism for applying the selected replica change to the Kubernetes workload. The project therefore separates the **decision logic** from the actual workload actuation through a dedicated actuator service.

The actuator exposes an HTTP endpoint that receives scaling requests from JanusScale. Each request contains information identifying the target namespace and Deployment, together with the desired replica count and the expected current replica state. This separation allows the analytical pipeline to generate a scaling decision without directly modifying the Kubernetes workload.

To protect the actuation endpoint, the implementation uses **HMAC-SHA256** request authentication. The actuator verifies the request signature before processing the scaling operation. In addition, the implementation maintains a whitelist of permitted workload targets, restricting actuation to explicitly configured Kubernetes resources.

The actuator also performs state-aware handling of scaling requests. If the requested replica count is already equal to the current replica count, the operation can be treated as a no-op rather than issuing an unnecessary scaling operation. The implementation uses per-deployment locking to serialize concurrent operations targeting the same workload.

After a scaling request is accepted, the actuator applies the requested replica count through the Kubernetes Deployment scaling interface and verifies the resulting workload state. The actuator also records scaling events in a JSONL audit log on a best-effort basis.

The resulting actuation path can be summarized as:

Decision → Authentication Request → Actuator → K8s Deployment Scale → Verification

This design separates **policy and decision-making** from **workload enforcement**, while providing authentication, workload restriction, concurrency control, and post-action verification. The implemented actuator provides application-level HMAC authentication; the project does not assume or claim transport-layer TLS from the actuator implementation itself.

2.7 Related Autoscaling Approaches

Existing autoscaling approaches can be broadly distinguished by the information used to trigger or determine scaling actions. **Reactive approaches** respond to the current observed system state, **event-driven approaches** use external workload or application events, while **predictive approaches** estimate future workload conditions before determining the required capacity.

Kubernetes HPA represents a reactive approach in which the desired replica count is determined from observed metrics. KEDA extends this model to event-driven workloads by integrating external event sources with Kubernetes autoscaling. These approaches provide established mechanisms for adapting workload capacity based on current or externally observed demand.

Predictive autoscaling approaches use historical time-series information to estimate future workload or resource demand. For example, Dang-Quang and Yoo investigated Bi-LSTM-based autoscaling for Kubernetes, while Yuan and Liao studied time-series-based elastic Kubernetes scaling. Liu et al. investigated workload forecasting for elastic resource management in an edge-cloud environment. These approaches illustrate how workload forecasting can be used to anticipate resource requirements rather than relying exclusively on the current system state.

JanusScale follows a different approach by using **multivariate anomaly detection as the trigger for an event-driven scaling pipeline**. MEWMA and T^2 form the central detection mechanism, while FFT-based temporal analysis and GARCH/DCC-related analysis provide supporting information about temporal behavior, variability, and dependencies. The resulting information is passed through attribution and decision stages before being enforced on the target Kubernetes workload.

Table 2.1: Comparison of Autoscaling Approaches.

Approach	Primary Information	Scaling Perspective
Kubernetes HPA	Current resource/custom metrics	Reactive
KEDA	External workload/application events	Event-driven
LSTM/Bi-LSTM forecasting	Historical time-series data	Predictive
JanusScale	Multivariate runtime anomalies + supporting temporal/dependency information	Anomaly-aware, event-driven

The main distinction between these approaches can therefore be summarized as follows: This comparison provides the context for the design choices made in JanusScale. The project does not attempt to establish that one autoscaling approach is universally superior to another. Instead, it focuses on implementing and evaluating an anomaly-aware closed-loop pipeline that combines multivariate detection with attribution, scaling decision, and controlled Kubernetes enforcement.

3. System Analysis and Requirements

3.1 System Context

JanusScale operates within a Kubernetes-based microservice environment in which the monitored workloads generate runtime telemetry and receive scaling actions from the autoscaling system. The system context therefore focuses on the interaction between JanusScale and the external components surrounding the online scaling workflow, rather than describing the internal implementation of each processing plane.

The main external entities are the **Kubernetes workloads**, the **telemetry sources**, the **Kubernetes API**, and the **calibration and evaluation environment**. Kubernetes workloads provide runtime observations that are collected by JanusScale for subsequent analysis. Based on the resulting analysis and scaling decision, JanusScale sends a scaling request through the actuation path to modify the replica count of the target Kubernetes workload.

The interaction can be summarized into two primary flows. First, runtime information flows from the monitored workloads and telemetry sources into JanusScale for analysis. Second, a scaling decision generated by JanusScale is transmitted to the Kubernetes environment, where the target workload capacity is updated. The resulting workload state then produces subsequent telemetry, forming the feedback required for the online scaling loop.

In addition to the online interaction, JanusScale has supporting calibration and evaluation activities. These activities provide parameters or candidate configurations for analysis and evaluation but are separated from the primary runtime scaling path. This separation allows the system context to distinguish between **online workload control** and **supporting offline analysis**.

Figure 3.1 presents the system context of JanusScale and its interactions with the surrounding Kubernetes environment. The internal processing stages of JanusScale are intentionally abstracted at this level; their organization into logical planes is described in **Section 4.1**.

3.2 Functional Requirements

The functional requirements of JanusScale are derived from the verified capabilities of the implemented online processing pipeline. The requirements focus on the functions required to transform runtime telemetry into a controlled Kubernetes scaling action, covering the logical planes **P1–P6**: ingestion, preprocessing, detection, attribution, decision, and enforcement.

The requirements are intentionally limited to capabilities supported by the inspected implementation and runtime evidence. Proposal-level features that have not been verified are not treated as implemented functional requirements.

Table 3.1: Functional Requirements Traced to Logical Planes.

Req. ID	Description	Related Plane (P1–P8)	Status (Implemented / Proposed)
FR-01	Receive and process runtime telemetry from monitored Kubernetes workloads.	P1 – Ingestion	Implemented
FR-02	Transform runtime telemetry into the implemented six-dimensional observation representation for downstream analysis.	P2 – Preprocessing	Implemented
FR-03	Detect abnormal multivariate runtime behavior using MEWMA and the associated T^2 statistic.	P3 – Detection	Implemented
FR-04	Perform supporting temporal analysis using FFT-derived features from selected runtime signals.	P2/P3 – Preprocessing/Detection	Implemented
FR-05	Attribute detected anomalous behavior to relevant signal dimensions and workload dependencies.	P4 – Attribution	Implemented
FR-06	Determine a scaling action and target replica count using the detected condition and available runtime context.	P5 – Decision	Implemented
FR-07	Apply runtime safety and policy conditions before automated actuation, including warmup, cooldown, freeze, and workload constraints.	P5/P8 – Decision / Operations & Safety	Implemented
FR-08	Transmit an authenticated scaling request to the configured actuator and apply the selected replica change to the target Kubernetes workload.	P6 – Enforcement	Implemented
FR-09	Protect the actuation interface through HMAC-SHA256 authentication and configured workload restrictions.	P6/P8 – Enforcement / Operations & Safety	Implemented

- UNIVERSITY OF INFORMATION TECHNOLOGY -

Req. ID	Description	Related Plane (P1–P8)	Status (Implemented / Proposed)
FR-10	Verify the resulting workload state after a scaling operation and record operational actuation events.	P6/P8 – Enforcement / Operations & Safety	Implemented
FR-11	Support calibration, candidate evaluation, and shadow evaluation without treating shadow candidates as production configurations.	P7 – Calibration & Evaluation	Implemented
FR-12	Feed the resulting workload state back into the telemetry pipeline to support closed-loop operation.	P1–P6	Demonstrated

The requirements above define the functional behavior of the implemented JanusScale system. **FR-12 is specifically supported by the demonstrated runtime scaling event in which the system detected an anomalous condition, attributed it to the frontend workload, generated a scale-out decision from 4 to 5 replicas, and successfully applied the corresponding actuation.** The requirement therefore describes the demonstrated closed-loop capability rather than implying a general claim of optimal autoscaling behavior.

Features described at the proposal level but not sufficiently verified by the implementation are excluded from the implemented requirements in this table. These include the proposed Fast/Slow GitOps workflow and other capabilities for which the available implementation evidence is insufficient.

3.3 Non-functional Requirements

The non-functional requirements define the operational, performance, security, reliability, and maintainability properties expected from the implemented JanusScale system. The requirements below are limited to properties supported by the inspected implementation and available runtime evidence.

Table 3.2: Non-Functional Requirements.

Req. ID	Description	Related Plane (P1–P8)	Status (Implemented / Proposed)
NFR-01	The online processing pipeline shall process runtime observations in periodic flush cycles, with the implemented runtime loop operating at approximately 3 seconds per flush cycle.	Performance	Implemented

- UNIVERSITY OF INFORMATION TECHNOLOGY -

Req. ID	Description	Related Plane (P1–P8)	Status (Implemented / Proposed)
NFR-02	The system maintains a bounded runtime observation window for online analysis rather than requiring unbounded historical data.	Performance / Resource Usage	Implemented
NFR-03	Scaling requests are authenticated using HMAC-SHA256 before being processed by the actuator.	Security	Implemented
NFR-04	The actuator shall restrict automated scaling to explicitly configured workload targets.	Security / Safety	Implemented
NFR-05	Concurrent scaling operations targeting the same deployment shall be serialized using per-deployment concurrency control.	Reliability	Implemented
NFR-06	A scaling request that already matches the current replica state shall not result in an unnecessary scaling operation.	Reliability / Efficiency	Implemented
NFR-07	The actuator verifies the resulting workload state after applying a scaling operation.	Reliability	Implemented
NFR-08	Operational scaling events record through the implemented JSONL audit mechanism on a best-effort basis.	Observability	Implemented
NFR-09	The system provides runtime safeguards that delay or prevent automated actuation under configured warmup, cooldown, freeze, or failure-handling conditions.	Safety	Implemented
NFR-10	The system support operation without requiring the analytical pipeline to directly modify Kubernetes workloads; workload changes shall be performed through the configured actuation interface.	Modularity / Safety	Implemented

The requirements above describe properties of the implemented system rather than general guarantees about Kubernetes autoscaling performance. In particular, the approximately three-second processing cycle describes the observed implementation loop and should not be interpreted as a guaranteed end-to-end actuation latency.

The project does not define transport-layer TLS as an implemented non-functional requirement because TLS is not established by the inspected actuator implementation itself. Similarly, the requirements do not claim a specific maximum CPU or memory overhead for the complete system; such measurements are treated as experimental results and are discussed in **Chapter 6**.

3.4 Constraints and Assumptions

The design and evaluation of JanusScale are subject to several constraints arising from the implemented system, the Kubernetes environment, and the available experimental evidence.

First, JanusScale is evaluated in a **Kubernetes-based microservice environment** with a configured telemetry pipeline and monitored workloads. The observations and experimental results presented in this report therefore describe the evaluated environment and should not be interpreted as universal performance characteristics across all Kubernetes workloads.

Second, the online analysis operates on a **bounded rolling observation window**. The implemented production configuration uses $W=240$ observations. This value represents the number of observations maintained by the analysis window and should not be interpreted directly as 240 seconds. The effectiveness of the analysis therefore depends on the availability of a sufficient recent observation history.

Third, the online pipeline assumes that the required runtime telemetry is available and sufficiently complete for processing. Missing or unsuitable telemetry can affect subsequent detection, attribution, decision-making, and enforcement.

Fourth, scaling decisions are subject to the **runtime constraints implemented by the system**, including replica limits and conditions such as warmup, cooldown, and freeze. Consequently, the detection of an anomalous condition does not imply that an immediate scaling action will always be issued.

Fifth, workload modification is performed through a **dedicated actuation interface** rather than by the analytical pipeline directly modifying the Kubernetes workload. The successful completion of the enforcement stage therefore depends on the availability of the actuator and the Kubernetes scaling interface.

Sixth, automated actuation is restricted to **configured workload targets**. The system is therefore not intended to provide unrestricted modification of arbitrary Kubernetes workloads.

Finally, the available experimental evidence constrains the claims that can be made about system performance. The project demonstrates an implemented closed-loop scaling workflow, but it does **not establish optimal autoscaling behavior or a universally optimal configuration**. Candidate configurations evaluated through shadow analysis are likewise treated as evaluation results rather than automatically as production configurations.

These constraints define the boundary of the system analysis and subsequent evaluation. Features proposed in the original system concept but not sufficiently supported by implementation evidence, such as the proposed Fast/Slow GitOps workflow, are treated as **intended or future capabilities rather than verified implementation features**.

3.5 Design Goals

The design of JanusScale is guided by the objective of connecting multivariate runtime analysis with controlled Kubernetes workload scaling. The system is designed around **MEWMA-based multivariate anomaly detection**, while temporal and dependency-related analysis provides supporting information for subsequent processing stages.

A key design goal is to maintain a clear separation between **detection, attribution, decision, and enforcement**. This separation allows an observed anomalous condition to be analyzed and associated with a relevant workload before a scaling action is generated and applied.

Another goal is to establish a **closed-loop scaling workflow**, in which runtime telemetry is processed to produce a scaling decision, the decision is enforced through a controlled actuation interface, and the resulting workload state becomes part of subsequent runtime observation.

The design also emphasizes **controlled actuation and operational safety**. Scaling requests are handled through a dedicated enforcement path with authentication, workload restrictions, and runtime safeguards. Calibration and evaluation activities are kept separate from the primary online scaling path so that experimental candidate configurations are not implicitly treated as production behavior.

Finally, the design is intended to remain **observable and experimentally verifiable**. The architecture and implementation should provide sufficient runtime evidence to trace a scaling event from detection through attribution, decision, and enforcement. The design therefore supports evaluation of the implemented closed loop without requiring a claim that the resulting autoscaling policy is globally optimal.

4. System Design and Architecture

4.1 Overall Architecture

JanusScale is organized as a set of logical processing planes that together form an event-driven Kubernetes autoscaling and self-healing workflow. The architecture separates the online processing path from calibration, evaluation, and operational safety functions, allowing each stage to have a distinct responsibility while remaining part of a common control loop.

As shown in **Figure 4.1**, the architecture consists of eight logical planes: **Ingestion (P1)**, **Preprocessing (P2)**, **Detection (P3)**, **Attribution (P4)**, **Decision (P5)**, **Enforcement (P6)**, **Calibration & Evaluation (P7)**, and **Operations & Safety (P8)**. These planes represent logical responsibilities rather than necessarily independent services or deployment units.

The primary online path follows **P1 → P2 → P3 → P4 → P5 → P6**. P1 receives runtime telemetry from the monitored Kubernetes environment. P2 transforms the incoming telemetry into the observation representation used by the analysis pipeline and generates supporting features. P3 performs multivariate anomaly detection, with **MEWMA and the associated T^2 statistic** serving as the central detection mechanism. P4 attributes detected anomalous behavior to relevant signal dimensions and workload dependencies. P5 uses the resulting information together with runtime context and configured constraints to determine the scaling action. P6 transfers the selected action to the configured actuator, which applies the replica change to the target Kubernetes workload.

P7 operates outside the primary online control path and supports calibration, candidate evaluation, and shadow analysis. Its outputs can provide configuration or evaluation information for the runtime system, but candidate configurations evaluated in shadow mode are not treated as production configurations solely as a result of the evaluation.

P8 is a cross-cutting logical plane that provides operational and safety-related functions across the online pipeline. These include runtime constraints and safeguards associated with automated decision-making and enforcement. P8 therefore does not represent another sequential processing stage after P6.

The architecture consequently separates the system into three complementary concerns: the **online scaling path (P1–P6)**, the **offline calibration and evaluation functions (P7)**, and the **cross-cutting operational and safety functions (P8)**. This organization provides the structural basis for the detailed data flow, control flow, and safety mechanisms described in the following sections.

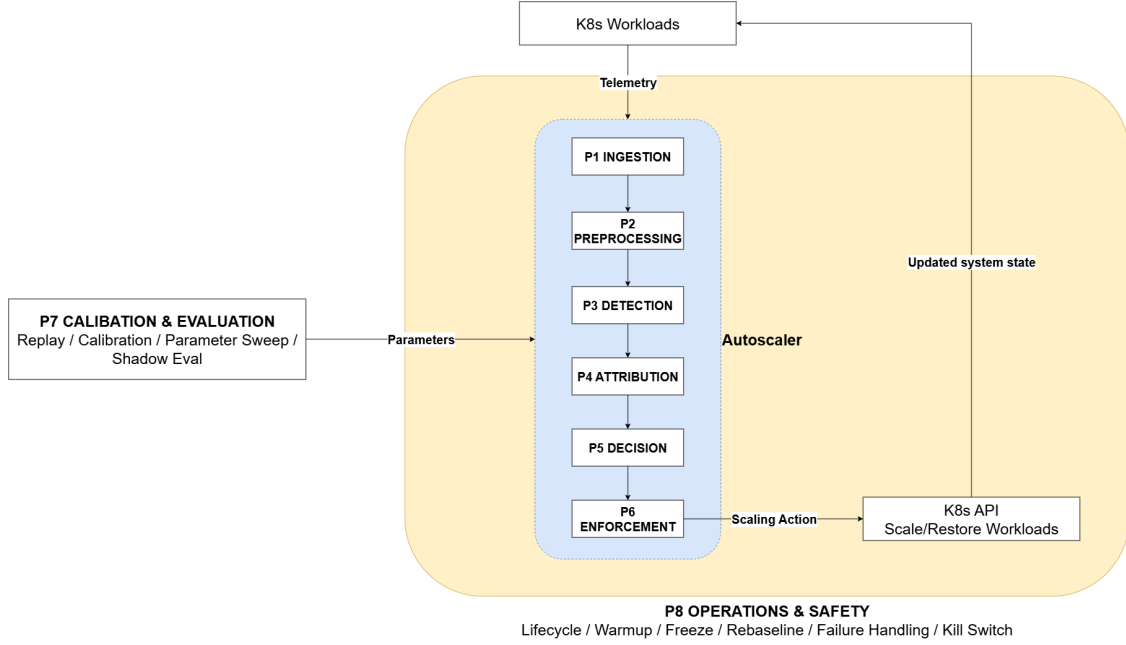


Figure 4.1: JanusScale Overall System Architecture

4.2 Telemetry-to-Decision Pipeline

The telemetry-to-decision pipeline describes how runtime observations are transformed into information that can be used to determine a Kubernetes scaling action. The pipeline covers the online processing stages from **telemetry ingestion through preprocessing, anomaly detection, attribution, and decision-making**, corresponding primarily to logical planes P1–P5.

At the ingestion stage, runtime telemetry from the monitored Kubernetes environment is received and organized for subsequent processing. The implementation processes telemetry associated with resource utilization and service behavior, including **CPU, memory, request rate (RPS), error behavior, ASYM, and SLOW**. These measurements are transformed into the six-dimensional observation representation used by the downstream analysis.

The preprocessing stage maintains a bounded rolling observation history. The implemented production configuration uses a window of **$W=240$ observations**. From the processed observations, the pipeline generates the representations required for subsequent analysis. FFT-based features are extracted for selected signals, while residual information is also generated for the multivariate analysis. RAM and error-rate signals remain in their direct representations rather than being processed through the FFT feature path.

The resulting observation and supporting features are then passed to the detection stage. **MEWMA and the associated T^2 statistic form the central multivariate detection mechanism**. The detector evaluates the current system behavior against the applicable

control limit and identifies anomalous conditions when the configured detection criteria are met. FFT-derived temporal information and GARCH/DCC-related information provide supporting context for the subsequent analysis.

When an anomalous condition is detected, the attribution stage analyzes the available residual and multivariate contribution information to identify the signal dimension associated with the detected deviation and relate it to the relevant workload dependency. This step converts a system-level anomaly into information that can be used by the decision stage.

The decision stage combines the attribution result with runtime context and configured operational constraints. Relevant context includes the current replica state, request rate, capacity-related information, and runtime conditions such as warmup, cooldown, freeze, and replica limits. Based on this information, the system determines whether an actuation should be issued and, when applicable, the desired replica count.

The resulting pipeline can therefore be summarized as:

$$\text{Telemetry} \rightarrow 6D \text{ Observation} \rightarrow \text{MEWMA}/T^2 \rightarrow \text{Attribution} \rightarrow \text{Decision}$$

Figure 4.2 illustrates this telemetry-to-decision flow. The enforcement stage is intentionally outside the boundary of this figure because the subsequent transfer of the decision to the actuator is described separately in the enforcement and closed-loop architecture sections.

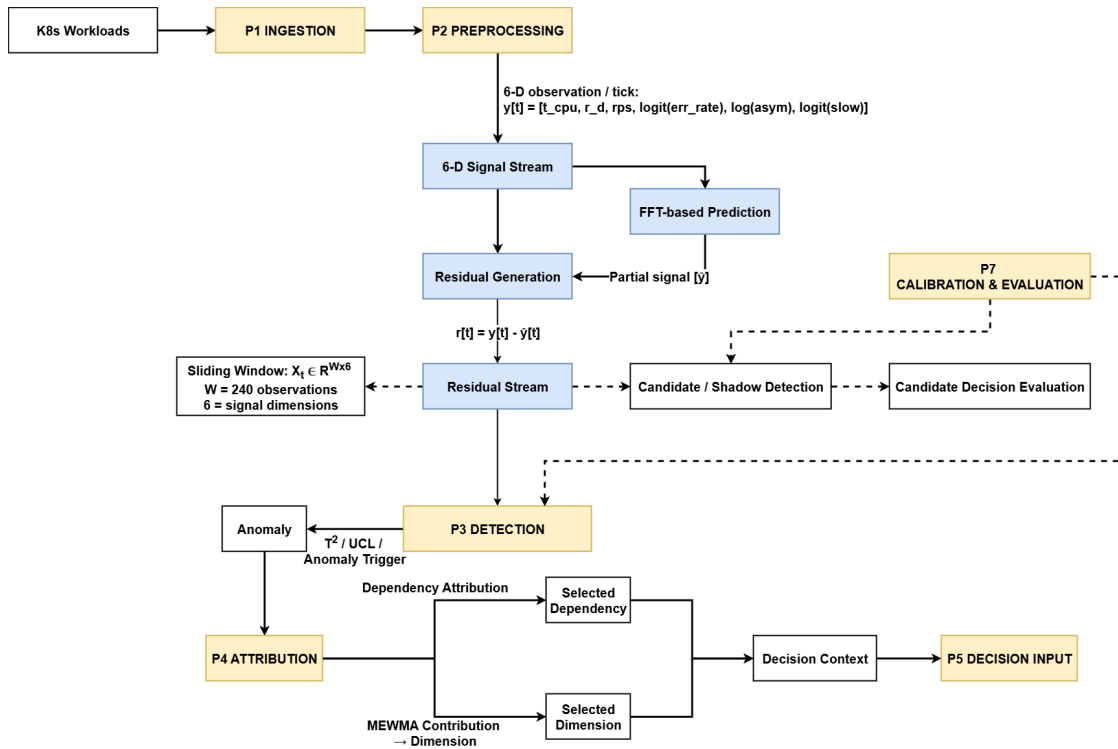


Figure 4.2: Telemetry-to-decision Data Flow

4.3 Decision and Enforcement Flow

The decision and enforcement flow describes how an analyzed runtime condition is converted into a controlled Kubernetes workload change. This flow connects the **Decision (P5)** and **Enforcement (P6)** planes and defines the boundary between the analytical pipeline and the Kubernetes environment.

After an anomalous condition has been detected and attributed, P5 evaluates the available runtime context before generating an actuation decision. The decision process considers information such as the **current replica count, request rate, capacity-related information, configured replica limits, and runtime safeguards** including warmup, cooldown, and freeze conditions. These conditions determine whether an identified anomaly should result in an actuation request and, when applicable, the desired replica count.

When an actuation is required, P5 generates a scaling request containing the target **namespace, Deployment, desired replica count, and expected current replica state**. The request is sent to the configured actuator rather than directly modifying the Kubernetes workload from the analytical pipeline.

P6 provides the enforcement boundary. The actuator authenticates incoming requests using **HMAC-SHA256** and restricts automated operations to configured workload targets. It also applies concurrency control for operations targeting the same deployment. If the requested replica count already matches the current state, the operation can be treated as a no-op.

For an accepted scaling operation, the actuator applies the requested replica count through the Kubernetes Deployment scaling interface and subsequently verifies the resulting workload state. The actuator records the resulting operation in its JSONL audit mechanism on a best-effort basis.

The resulting flow can be summarized as:

*Anomaly + Attribution → P5 Decision → Authentication Request
→ P6 Actuator → Kubernetes Scale → State Verification*

The enforcement result then becomes part of the subsequent workload state and telemetry, allowing the overall system to operate as a closed loop. This flow therefore separates **analytical decision-making** from **workload enforcement**, while maintaining a controlled boundary for automated scaling.

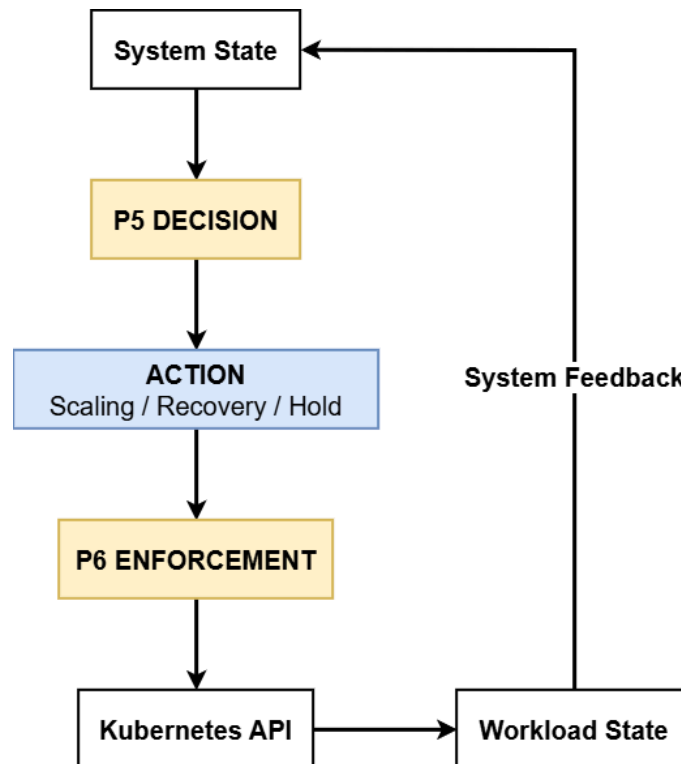


Figure 4.3: Decision and Recovery/Enforcement Flow

4.4 Calibration and Evaluation Architecture

JanusScale separates **calibration and evaluation activities** from the primary online scaling path. These activities are represented by the logical **Calibration & Evaluation plane (P7)** and provide supporting information for configuring and assessing the runtime analysis pipeline without becoming another sequential stage in the online decision flow.

The calibration process uses available historical or experimental runtime data to derive parameters required by the analysis pipeline. These parameters can include detection-related thresholds and configuration values used by the runtime system. The purpose of this stage is to establish configurations that can subsequently be evaluated under controlled conditions before being used in the online pipeline.

In addition to calibration, JanusScale supports **candidate evaluation and shadow analysis**. Candidate configurations can be evaluated against the available experimental data or runtime observations without treating the candidate itself as an automatically deployed production configuration. This separation allows alternative configurations to be compared with the runtime configuration while keeping the production control path unchanged.

The evaluation results provide evidence about the behavior of the candidate or calibrated configuration, including detection characteristics and other measured properties available from the experiment. These results can then be used as supporting information when selecting a configuration for runtime operation.

The architecture can therefore be summarized as:

*Historical/Experimental Data → Calibration → Candidate Configuration
→ Shadow Evaluation → Evaluation Evidence → Runtime Configuration*

This flow is intentionally separated from the online path **P1 → P2 → P3 → P4 → P5 → P6**. P7 supports the configuration and evaluation of the system, while the actual production scaling decision remains within the online processing path.

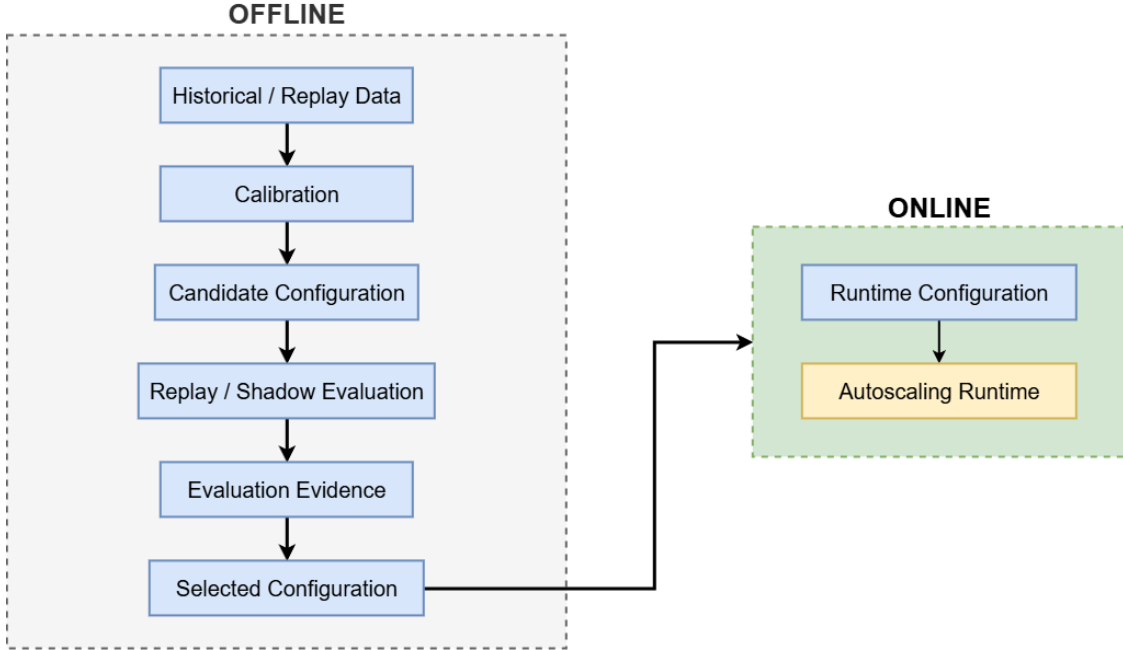


Figure 4.4: Calibration & Evaluation Architecture

4.5 Data Flow

The JanusScale data flow describes how runtime telemetry is transformed into analytical representations and decision information across the logical planes. The main online data path proceeds from **P1 to P6**, while P7 provides a separate calibration and evaluation branch and P8 provides cross-cutting operational and safety information.

At P1, the system receives runtime telemetry from the monitored Kubernetes environment. The telemetry includes **CPU, RAM, RPS, error behavior, ASYM, and SLOW**. During preprocessing, these signals are transformed into the six-dimensional observation used by the multivariate analysis: `err_rate`

$$y_t = [t_{cpu}, r_d, RPS, \text{logit}(\text{err_rate}), \log(ASYM), \text{logit}(SLOW)]^T.$$

The preprocessing stage also generates supporting representations for the downstream analysis. **CPU, RPS, ASYM, and SLOW** can contribute to the FFT-based processing path, while **RAM and error-rate information remain in their direct representations**. The resulting observation history and derived features are then used by the detection and attribution stages.

P3 produces multivariate detection information using **MEWMA** and T^2 . When an anomalous condition is detected, P4 uses the available residual and contribution information to associate the anomaly with relevant signal dimensions and workload dependencies. P5 then combines this information with runtime context to produce a scaling decision, which is passed to P6 for controlled workload enforcement.

P7 follows a separate data path for **calibration, candidate evaluation, and shadow analysis**. Data and configurations used in this branch support evaluation of the analysis pipeline but are not treated as part of the sequential online P1–P6 processing path. P8 does not introduce a separate data-processing stage; instead, it provides operational and safety information that constrains or accompanies the online processing and enforcement flow.

The resulting data mapping is summarized in Table 4.1.

Table 4.1: Data Mapping by Logical Plane.

Plane	Input Data	Output Data	Next Destination
P1 - Ingestion	Runtime telemetry: CPU, RAM, RPS, ERR, ASYM, SLOW	Organized runtime telemetry	P2
P2 - Preprocessing	Raw/normalized telemetry	6-D observation, residuals, FFT-derived features	P3 / P4
P3 - Detection	6-D observation, MEWMA state, supporting features	MEWMA/ T^2 detection result, anomaly information	P4
P4 - Attribution	Detection result, residual/contribution information, dependency information	Anomalous dimension and workload attribution	P5
P5 - Decision	Attribution + runtime context + replica/capacity state + safety constraints	Scaling decision and desired replica count	P6
P6 - Enforcement	Authenticated scaling request	Applied replica state, verification result, audit event	Kubernetes workload → P1
P7 - Calibration & Evaluation	Historical/experimental data, candidate configurations	Calibrated parameters, candidate/shadow evaluation evidence	Runtime configuration / evaluation
P8 - Operations & Safety	Runtime constraints, operational state, safety conditions	Constraints/safeguards applied across online flow	P1–P6

4.6 Control Flow

The control flow of JanusScale describes how the system progresses from an observed runtime condition to a scaling action and subsequently returns to the monitoring state. Unlike the data flow in Section 4.5, which focuses on the transformation of telemetry and analytical information, the control flow emphasizes **trigger conditions, decision points, and system actions**.

During normal operation, JanusScale continuously processes incoming telemetry through the online pipeline. When the detection stage identifies an anomalous condition according to the configured MEWMA/ T^2 criteria, the system proceeds to attribution. If no anomaly is detected, the current processing cycle does not generate an automated scaling action and the system continues monitoring subsequent observations.

When an anomaly is detected, the attribution stage determines the relevant signal dimension and workload dependency. The resulting information is passed to the decision stage, where the system evaluates the current workload state and configured runtime constraints. Conditions such as **warmup, cooldown, freeze, replica limits, and available capacity** can affect whether the system proceeds with actuation.

If the decision stage determines that no scaling action should be issued, the system returns to the monitoring state. If an action is required, P5 generates the corresponding scaling request and passes it to the enforcement stage. P6 authenticates and validates the request before applying the requested replica change through the configured Kubernetes actuation interface.

After enforcement, the resulting workload state is verified. The updated workload state subsequently produces new telemetry, which re-enters the ingestion stage. This feedback path completes the online control loop.

The control flow can therefore be summarized as:

Monitor → *Detect* → *Attribute* → *Evaluate Constraints*
→ {*No Action* → *Monitor*; *Scale* → *Enforce* → *Verify* → *Monitor*}

This control flow provides the decision logic connecting anomaly detection with workload enforcement while allowing runtime safeguards to prevent or delay actuation when the configured conditions are not satisfied.

4.7 Safety and Operational Controls

JanusScale includes a set of operational and safety controls that constrain the behavior of the online scaling pipeline. These controls are represented by the cross-cutting **Operations & Safety plane (P8)** and interact primarily with the decision and enforcement stages.

The runtime configuration includes **cooldown and freeze mechanisms** that can constrain automated actuation. The evaluated configuration sets JANUS_COOL=100, JANUS_FRZ=1, JANUS_FRZ_REL=8, JANUS_FRZ_MAX=400, JANUS_FRZ_DEP=1, and JANUS_FRZ_ACT=0. These parameters configure the runtime conditions under which automated scaling can be restricted or delayed.

The system also uses an actuation warmup mechanism. In the evaluated test configuration, `JANUS_ACT_WARMUP_TICKS=5` was explicitly configured for faster testing. The configuration comment indicates that the default value is 720 ticks, corresponding to 3W. Therefore, the value of 5 should be understood as a **test configuration**, rather than as the normal production warmup setting.

Actuation failure handling is also configured in the runtime pipeline. The runtime logs report `fail_max=3` when the actuator is enabled, indicating that three failures constitute the configured maximum for the corresponding failure-handling mechanism. The implementation supports repeated actuation failures contributing to a **dependency kill-switch**. However, the available runtime logs do not provide sufficient evidence to claim that the kill-switch was actually triggered during the evaluated runs.

The implementation also contains rebaseline-related handling for dependency states. However, the available runtime output does not by itself establish a specific observed rebaseline event. Therefore, rebaseline is treated here as an implemented operational mechanism rather than as an experimentally demonstrated event.

Overall, the safety controls can be viewed as constraints around the online decision and enforcement process:

Runtime State → Warmup/Cooldown/Freeze Checks → Decision → Actuation → Failure Handling

These mechanisms are intended to constrain automated operation under configured runtime conditions. The reported parameter values describe the evaluated configuration and are not presented as universally optimal safety settings.

5. Implementation

5.1 P1 - Ingestion

The **P1 Ingestion** plane is implemented as the runtime telemetry collection layer of JanusScale. The main ingestion service is implemented using **FastAPI** and exposes a `POST /` endpoint. The service binds on `0.0.0.0:5001`, while the runtime processing loop operates with a configured **3-second flush interval** (`FLUSH_SEC = 3.0`).

The ingestion endpoint receives the request body as a compressed binary payload. The implementation decompresses the payload using **Snappy** and then parses the resulting binary representation using the internal varint and timestamp parsing routines. For each telemetry sample, the service extracts the metric value, timestamp, and associated labels before routing the sample according to its metric type.

```
@app.post('/')
async def ingest(request: Request):
    global ring_write
    body = await request.body()
    recv = time.perf_counter_ns()
    raw = bytes(cramjam.snappy.decompress_raw(body))
    p = 0
    while p < len(raw):
        tag, p = read_varint(raw, p)
```

```
f = tag >> 3
w = tag & 7
if f == 1 and w == 2:
    l, p = read_varint(raw, p)
    value, ts = parse_ts(raw[p:p+1], _lbl)
    p += 1
    ...
```

Four telemetry paths are handled by the ingestion implementation. **cAdvisor** metrics provide CPU and memory-related measurements at pod level. **Hubble HTTP request metrics** are used to derive request-rate and error-rate information. **Hubble HTTP request-duration bucket metrics** provide the latency distribution used later to derive slow-request and percentile-related measurements. The implementation also processes an **Envoy upstream-request metric** for additional service-level telemetry.

For cAdvisor samples, the implementation filters data by the configured namespace `bank-of-anthos`, identifies the corresponding pod, and stores the measurement in a sharded per-pod state structure. The ingestion state uses **16 shards**, while the runtime ring buffer has a configured capacity of **8192 entries**. This state is subsequently consumed by the periodic processing loop.

Counter-based telemetry is not treated as if every sample were separated by a fixed interval. Instead, the implementation calculates rates from the difference between successive counter values and their timestamps. The processing logic applies timestamp-gap limits and tracks fresh, invalid, and excessive-gap observations. For per-pod measurements, the configured maximum timestamp gap is **45 seconds**. Measurements exceeding the applicable gap limit are excluded from rate calculation rather than being treated as normal samples.

The ingestion state is periodically consumed by `flush_loop()`. At each flush, stale pod entries are removed after the configured number of stale cycles, telemetry from the different metric stores is swept and converted into runtime rates, and deployment-level quantities are aggregated. The resulting values include CPU, RAM-related change, RPS, error rate, asymmetry, and slow-request ratio.

These measurements are then assembled into the six-dimensional observation used by the downstream analytical pipeline:

$$y = [t_{cpu}, r_d, RPS, \text{logit}(err_rate), \log(ASYM), \text{logit}(SLOW)],$$

The implementation maintains explicit validity flags for the six dimensions. When a measurement is temporarily unavailable or fails its validity condition, the previous valid value is retained instead of propagating an invalid observation into the downstream pipeline.

The ingestion endpoint itself does **not** perform anomaly detection or workload scaling. After processing the incoming payload, it records ingestion timing information in the internal ring buffer and returns an HTTP 200 response. The subsequent `flush_loop()` performs the periodic transformation from accumulated telemetry into the runtime observation consumed by P2 and P3.

Therefore, the implementation boundary of P1 can be summarized as:

*Compressed Telemetry → Decompression & Parsing → Metric Routing
→ pod/metric state → Timestamp aware Aggregation → 6D Observation → P2/P3*

Although the same `ingester.py` process also contains preprocessing, MEWMA/ T^2 detection, attribution, decision, and actuation logic, these functions are treated as separate **logical planes** in the JanusScale architecture. P1 is specifically responsible for receiving and organizing runtime telemetry before it enters the downstream analytical pipeline.

5.2 P2 - Preprocessing

The **P2 Preprocessing** plane transforms the telemetry-derived observation from P1 into signals suitable for downstream anomaly detection. Its main responsibilities are handling observation validity, estimating expected temporal components for selected dimensions, and generating residual signals. The implementation keeps the six-dimensional observation structure introduced by P1 while applying different preprocessing paths depending on the characteristics of each dimension.

Before the observation is passed to the detector, the implementation evaluates the validity of each dimension. The validity conditions depend on the availability and quality of the corresponding telemetry source. When a dimension is temporarily invalid, the implementation retains its most recent valid value through the `last_valid` state rather than directly propagating an invalid measurement into the downstream pipeline. This allows the detector to continue operating while explicitly preserving the most recent valid observation.

The preprocessing stage also applies an FFT-based harmonic model to selected dimensions of the observation. The implementation defines the FFT input dimensions as **CPU, RPS, ASYM, and SLOW**, while **RAM and ERR** are retained as direct observations without an FFT-derived expected component. For each FFT-enabled dimension, the active harmonic model produces an expected value at the current time, denoted as $\hat{y}_t$. The corresponding residual is then calculated as:

$$r_t = y_t - \hat{y}_t$$

The implementation performs this transformation explicitly for the active harmonic models:

```
for fi in range(len(FFT_DIM_IDX)):
    di = FFT_DIM_IDX[fi]
    a = hset.active[fi]
    yh[di] = (
        a.at(now_wall - hset.t0a[fi]) * FFT_EVAL_SCALE[fi]
        if a.k else 0.0
    )
    res[di] = y[di] - yh[di]
yh[1] = 0.0
res[1] = y[1]
yh[3] = 0.0
res[3] = y[3]
```

Here, y_h represents the estimated expected signal and res represents the resulting residual. The explicit assignments for the RAM and ERR positions ensure that these dimensions are not assigned an FFT-based expected component.

The harmonic models used at runtime are obtained from the FFT fitting process. The implementation's `fft_fit()` procedure first applies robust preprocessing based on the **median and median absolute deviation (MAD)**, followed by trend removal and frequency-domain analysis. Candidate spectral peaks are selected and grouped before constructing the harmonic representation. The fitting procedure also applies configured limits on the number of retained harmonics and a variance-explained criterion before a model is accepted.

For longer-term operation, the implementation periodically updates the FFT models using historical telemetry obtained from the monitoring backend. The slow fitting path applies coverage, gap, trend, and variance-explained checks before replacing the active harmonic model. This separates the process of fitting an expected temporal pattern from its use during online detection.

The output of P2 is therefore not a new independent feature vector, but a combination of the validated six-dimensional observation and its corresponding expected-signal and residual representations. These residuals are subsequently consumed by the P3 detection stage, where $MEWMA/T^2$ evaluates whether the current observation deviates sufficiently from the learned runtime behavior.

The preprocessing boundary can therefore be summarized as:

*6D Observation → Validity Handling → Selected Dimension FFT Modeling
→ Expected Signal → Residual Generation → P3 Detection*

The FFT component is treated as part of preprocessing rather than as an independent architectural plane. This reflects the implementation structure: FFT is used to estimate expected temporal behavior and construct residuals, while the actual multivariate anomaly decision is performed downstream by the $MEWMA/T^2$ detector.

5.3 P3 - Detection

The **P3 Detection** plane applies the multivariate anomaly detection method to the residual signals produced by P2. Its primary detector is a **MEWMA-based multivariate monitoring process**, with the resulting statistic evaluated against a configured upper control limit (UCL). The detector operates on the six-dimensional residual representation while preserving the temporal state required by the sequential monitoring procedure.

The implementation maintains a MEWMA state over the current observation stream. Given the residual vector r_t , the smoothed monitoring state is updated recursively as:

$$z_t = \lambda r_t + (1 - \lambda)z_{t-1},$$

where λ is the configured smoothing parameter. In the current implementation, the MEWMA dimension is **6** and the configured smoothing parameter is $\lambda = 0.2$.

The implementation then computes a multivariate quadratic monitoring statistic from the MEWMA state:

$$T_t^2 = z_t^T \Sigma_z^{-1} z_t,$$

where Σ_z represents the covariance structure used by the detector. The resulting T^2 statistic is compared with the configured UCL. A detection event is generated when the monitoring statistic exceeds the applicable threshold and satisfies the additional detection-state conditions.

A simplified representation of the runtime sequence is:

```
z = mewma.update(res)
t2 = mewma.t2(z)
if t2 > ucl:
    detection = True
else:
    detection = False
```

The actual implementation also maintains detector state across ticks rather than treating each observation independently. This is important because MEWMA is a sequential monitoring method: the current statistic depends on the accumulated monitoring state from previous observations.

The detector uses the **240-sample rolling window** configured for the MEWMA pipeline. This value represents the number of observations retained by the monitoring process; it should not be interpreted as a fixed number of seconds because the runtime flush interval and telemetry timing are separate configuration parameters.

The implementation also applies the configured detection controls around the primary T^2 statistic. The runtime configuration includes:

- **MEWMA dimension:** 6
- **MEWMA smoothing parameter:** $\lambda = 0.2$
- **MEWMA window:** 240 observations
- **Univariate threshold factor:** JANUS_UNI_K=3.0
- **Secondary univariate threshold factor:** JANUS_UNI_K2=2.0
- **Cooldown/freeze controls:** handled downstream as part of the decision and safety logic.

The univariate statistics are used as supporting detection information rather than replacing the six-dimensional MEWMA/ T^2 detector. Runtime evidence records both the multivariate statistic and per-dimension diagnostic values, allowing a detected event to be subsequently attributed to a particular signal dimension.

For example, a runtime detection record contains information of the following form:

```
UNI trig=1 n_k3=2 n_k2=2 max=ram z=22.678 mewma=1 t2=4958.377
D cpu=1.749 ram=4658.511 rps=5.423 err=21.176 asym=51.639
slow=19.940
```

This representation separates the **multivariate detection result** from the dimension-level diagnostic statistics. The latter are subsequently consumed by P4 Attribution.

The P3 output therefore consists of:

1. the updated MEWMA monitoring state;
2. the current T^2 statistic;
3. the applicable detection threshold/UCL;
4. detection state and supporting univariate diagnostics;
5. the residual and dimension-level information required by P4.

The logical boundary of P3 is:

*P2 Residuals → MEWMA State Update → T^2 Calculation
→ Threshold Evaluation → Detection Event → P4 Attribution*

P3 is therefore responsible for answering **whether the current multivariate observation is anomalous**. Determining **which workload or signal dimension is responsible** is handled separately by P4, while deciding whether an operational action should follow the detection is handled by P5.

5.4 P4 - Attribution

The **P4 Attribution** plane identifies the signal dimension and workload dependency associated with an anomaly detected by P3. Its purpose is to transform the multivariate detection result into actionable diagnostic information that can be consumed by the decision layer.

The implementation evaluates the dimension-level diagnostic values generated alongside the $MEWMA/T^2$ statistic. The six monitored dimensions are **CPU, RAM, RPS, error rate, asymmetry, and slow-request ratio**. These values provide the basis for identifying which component contributes most strongly to the detected anomaly.

A runtime attribution event records the selected dimension, its diagnostic value, the applicable threshold, the detection significance, and the associated dependency. A representative runtime record is:

```
ATTR dim=ram d=4658.511 thr=9.149 sig=1 rule=rat dep=frontend z=+20.761
```

Here, `dim=ram` identifies RAM as the attributed dimension, while `dep=frontend` associates the anomaly with the `frontend` deployment. The diagnostic value and threshold are retained in the event for subsequent decision processing.

The attribution stage also uses dependency-level runtime state maintained by the system. The implementation updates dependency statistics during the aggregation cycle and associates the resulting observations with the corresponding workload dependency. This allows the detector output to be interpreted in terms of a concrete Kubernetes workload rather than only as an abstract six-dimensional anomaly.

The resulting attribution can therefore be represented as:

Detection → Dimension Contribution → Dependency Mapping → Attributed Anomaly

For the runtime closed-loop event used in the evaluation, P3 detected a multivariate anomaly and P4 attributed the dominant signal to **RAM** and the affected dependency to **frontend**. The subsequent decision record used this attribution when determining the scaling action:

```
DECIDE dep=frontend dim=ram cur=4 desired=5 max=20 capr=16.05 rps=69.0  
run=18 act=scale_out
```

This establishes the interface between P4 and P5: P4 provides the **diagnostic context**, while P5 evaluates the workload state, replica constraints, policy conditions, and other decision inputs before determining whether an action should be issued.

P4 does not directly modify the Kubernetes workload. Its output is consumed by the decision layer, preserving the separation between **anomaly attribution** and **operational actuation**.

The logical boundary of P4 is:

P3 Detection → Dimension Level Attribution → Dependency Mapping

→ *Attributed Anomaly* → *P5 Decision*

Thus, P4 answers “**which monitored dimension and workload dependency are associated with the detected anomaly?**”, while P5 is responsible for determining **whether that information should result in a scaling action**.

5.5 P5 - Decision

The **P5 Decision** plane converts the attributed anomaly from P4 into an operational decision by evaluating the current workload state, replica state, capacity constraints, and runtime safety policies. Unlike P4, which provides diagnostic information, P5 determines whether a scaling action should be issued and, if so, computes the desired replica count.

The decision logic receives the attributed dependency and dimension together with runtime context such as the current replica count, observed request rate, estimated capacity, configured replica limits, cooldown state, freeze state, and other policy conditions.

The implementation explicitly considers the current and desired replica counts. A representative runtime decision record is:

```
DECIDE dep=frontend dim=ram cur=4 desired=5 max=20 capr=16.05 rps=69.0  
run=18 act=scale_out tax=3 dcc=0 ACTUATED
```

This record shows that the `frontend` dependency was associated with a RAM anomaly, the current replica count was 4, and the calculated desired replica count was 5. The decision also retained the configured maximum replica limit and runtime capacity/request information.

The decision layer is additionally constrained by runtime safety controls. The current configuration includes a **cooldown value of 100**, together with freeze-related controls:

```
JANUS_COOL=100  
JANUS_FRZ=1  
JANUS_FRZ_REL=8  
JANUS_FRZ_MAX=400  
JANUS_FRZ_DEP=1  
JANUS_FRZ_ACT=0
```

These controls are evaluated as part of the decision/safety process rather than being treated as properties of the anomaly detector itself. The freeze mechanism can temporarily prevent scaling decisions under the configured conditions, while the maximum freeze duration bounds a continuous freeze interval. The implementation also contains rebaseline behavior when the maximum freeze duration is reached.

The decision layer also supports an **action warm-up mechanism**. The current test configuration uses:

```
JANUS_ACT_WARMUP_TICKS=5
```

This value is explicitly a test configuration; the runtime configuration comments indicate that the normal default is restored before production use. Therefore, the value should be reported as an experimental configuration rather than as a fixed production policy.

Before an action is passed to P6, P5 also evaluates workload-specific constraints and avoids issuing a scaling request when the resulting replica state does not require a change. This separates **decision generation** from **enforcement**: P5 determines the desired state, while P6 is responsible for applying that state to Kubernetes.

The runtime closed-loop evidence demonstrates this interface. For the observed event, P5 generated:

```
cur=4 desired=5
```

and the subsequent actuator log confirmed the transition from 4 to 5 replicas. This provides evidence for the **P5 → P6** handoff and successful enforcement for that event.

The logical boundary of P5 is:

*P4 Attributed Anomaly → Workload State → Policy & Safety Evaluation
→ Desired Replica Count → P6 Enforcement*

Thus, P5 answers “**given the detected and attributed condition, current workload state, and configured policies, should the system request a replica-state change, and what target should it request?**” It does not directly modify the Kubernetes workload; that responsibility belongs to P6.

5.6 P6 - Enforcement

The **P6 Enforcement** plane applies the desired replica state generated by P5 to the Kubernetes workload. In the current implementation, enforcement is separated into a dedicated actuator service rather than having the analytical pipeline directly modify Kubernetes resources.

The actuator exposes a **FastAPI /scale endpoint** and receives scaling requests from JanusScale. Requests are authenticated using an **HMAC-SHA256 signature**. The actuator also restricts requests to configured workload targets, including the `bank-of-anthos/frontend` deployment used in the runtime evaluation.

The actuator uses the Kubernetes **Deployment scale subresource** to change the replica count. A per-deployment asynchronous lock serializes concurrent scaling operations for the same workload. When the requested replica count is already equal to the current state, the actuator can treat the request as a no-op rather than issuing an unnecessary scaling operation.

The request authentication is based on HMAC-SHA256. Conceptually, the actuator verifies a signature derived from the request payload before accepting the scaling operation:

```
signature = HMAC-SHA256(secret, request_body)
```

- UNIVERSITY OF INFORMATION TECHNOLOGY -

The actuator also performs workload restrictions before applying the requested state. This prevents an authenticated request from being treated as permission to modify an arbitrary Kubernetes deployment.

After requesting the replica change, the implementation performs **post-action verification** against the workload state. The actuator records the requested and observed replica states in its audit output. The audit mechanism uses JSONL records and is treated as best-effort logging rather than as the source of enforcement itself.

A runtime event provides evidence of the complete P5-to-P6 transition:

```
DECIDE dep=frontend dim=ram cur=4 desired=5 ...  
ACT ok dep=frontend namespace=bank-of-anthos desired=5 applied_from=4  
applied_to=5
```

This demonstrates that the desired state generated by P5 was passed to the actuator and that the actuator reported a successful transition from **4 to 5 replicas** for the `frontend` deployment.

The implementation also contains actuator failure protection. Repeated actuator failures are counted, with the configured failure limit set to **3**. The failure mechanism can trigger the configured actuator kill-switch after repeated failures. The available runtime evidence does not establish that this failure threshold was reached during the evaluated event, so it is described here as an implemented safeguard rather than an observed failure scenario.

The current application-layer actuator uses HTTP for the request path; **TLS is not implemented in the inspected actuator layer**. Therefore, the HMAC mechanism should be described specifically as request authentication/integrity protection rather than as a replacement for transport-layer security.

The logical boundary of P6 is:

P5 Desired State → Authenticated Actuation Request → Target Validation
Kubernetes Scale Operation → State Verification → Audit

P6 therefore provides the enforcement boundary between JanusScale's analytical/decision pipeline and the Kubernetes workload. It is responsible for **applying and verifying the requested replica state**, rather than determining whether the workload should scale in the first place.

5.7 P7 - Calibration and Evaluation

The **P7 Calibration and Evaluation** plane operates outside the online control loop. Its purpose is to evaluate detector and policy configurations using historical or replayed telemetry before a configuration is used by the runtime pipeline. The resulting configurations are treated as **candidates** until they are evaluated against the available calibration and evaluation evidence.

The candidate configuration file contains multiple parameter sets rather than representing a single automatically selected production configuration. For example, `eval.json` contains candidate configurations **B** and **C**, with different UCL, latch, cooldown, and dimension-level diagnostic thresholds.

The candidate records include values such as:

```
B:
  ucl = 56.5673
  latch = 6
  cool = 100
  dthr = [25.6655, 6.5573, 6.9970, 6.8730, 29.1200, 20.6941]

C:
  ucl = 50.0334
  latch = 8
  cool = 100
  dthr = [24.9689, 6.5488, 6.9570, 6.8484, 27.9837, 20.1079]
```

These candidates are evaluated through separate offline experiments recorded in the `out_*.txt` files. The evaluation covers different aspects of the detector and policy configuration, including false-alarm behavior, fitted temporal structure, latch/freeze behavior, operating-window calibration, and supporting univariate detection behavior.

The available evaluation outputs include:

- `out_ev3.txt` - multivariate detection evaluation;
- `out_far.txt` - false-alarm/grid evaluation;
- `out_fit.txt` - temporal fitting and freeze-related evaluation;
- `out_latch.txt` - latch/cooldown evaluation;
- `out_op.txt` - operating-window calibration;
- `out_uni2.txt` - univariate supporting evaluation.

The offline process therefore does not directly alter the online controller during each experiment. Instead, it produces quantitative evidence that can be used to compare candidate parameterizations and determine which configuration is suitable for subsequent runtime evaluation.

An important distinction is maintained between **candidate evaluation** and **runtime deployment**. The presence of a candidate in `eval.json`, or a favorable measurement in one of the `out_*.txt` experiments, does not by itself establish that the candidate is the optimal configuration or that it was automatically promoted to production. The available evidence

supports describing these configurations as **evaluated candidates** unless the corresponding runtime configuration and selection step are explicitly documented.

The P7 output is therefore a parameter/configuration set accompanied by its evaluation evidence:

*Replay / Historical Data → Calibration → Parameter Sweep
→ Candidate Configurations → Shadow Evaluation → Evidence → Runtime Configuration*

P7 is deliberately outside the P1–P6 online loop. Its role is to provide calibration and evaluation evidence without directly participating in per-tick anomaly detection, decision making, or Kubernetes actuation.

5.8 P8 - Operations and Safety

The **P8 Operations and Safety** plane provides cross-cutting runtime controls around the online JanusScale pipeline. It does not constitute an additional processing stage between P7 and the controller. Instead, its controls constrain detection, decision, and actuation behavior to reduce unsafe or unnecessary workload changes.

The implementation includes several runtime safeguards covering cooldown, freeze, warm-up, actuator failure handling, and rebaseline behavior. The main runtime configuration contains:

```
JANUS_COOL=100
JANUS_FRZ=1
JANUS_FRZ_REL=8
JANUS_FRZ_MAX=400
JANUS_FRZ_DEP=1
JANUS_FRZ_ACT=0
JANUS_ACT_WARMUP_TICKS=5
```

The **cooldown** mechanism limits repeated scaling decisions according to the configured runtime policy. Freeze controls provide an additional mechanism for temporarily suppressing scaling decisions under configured conditions. `JANUS_FRZ_MAX=400` bounds the duration of a continuous freeze interval; the implementation also contains rebaseline behavior when this maximum is reached.

The actuator provides a separate operational safeguard through failure counting. The actuator's configured failure limit is **3 consecutive failures**, after which the corresponding kill-switch mechanism can prevent further actuation. This is a protection mechanism implemented at the enforcement boundary; the available runtime evidence does not establish that the failure limit was reached during the evaluated closed-loop event.

The implementation also uses workload-level serialization at the actuator. A per-deployment lock prevents concurrent scaling operations for the same deployment from being applied simultaneously. Combined with target whitelisting and request authentication, this provides an enforcement boundary between the analytical pipeline and Kubernetes.

The runtime system also performs stale-state handling in the ingestion/processing path. Pod state that remains unseen for the configured number of cycles is removed from the active state. Timestamp-based gap checks additionally prevent excessively delayed counter samples from being treated as ordinary continuous-rate observations.

The current implementation therefore separates **analytical processing**, **decision policy**, and **enforcement safeguards**. P8 does not independently generate anomaly detections or scaling targets; instead, it constrains how the online system operates when telemetry is incomplete, repeated decisions occur, freeze conditions are active, or actuation failures occur.

The logical role of P8 is:

*Runtime State & Safety Conditions → Constrain Detection/Decision/Actuation
→ Controlled Closed Loop*

This separation is important for the implementation boundary: P8 represents the **operational and safety controls surrounding the JanusScale loop**, rather than an additional “recovery” processing plane.

5.9 Configuration and Runtime Parameters

The JanusScale runtime configuration is defined through environment variables in `run.sh`. These parameters control the online decision policy, freeze/cooldown behavior, detector thresholds, FFT fitting, calibration intervals, debugging, and actuator endpoint.

The verified runtime parameters are summarized below.

Table 5.1: run.sh Configuration Parameters.

Variable	Value	Meaning	Note
JANUS_COOL	100	Cooldown parameter controlling the minimum interval between applicable scaling decisions	Runtime configuration
JANUS_BAND	0.2	Configured decision band/tolerance used by the runtime policy	Runtime configuration
JANUS_FRZ	1	Enables the freeze mechanism	Runtime configuration
JANUS_FRZ_REL	8	Freeze-release parameter	Runtime configuration
JANUS_FRZ_MAX	400	Maximum duration of one continuous freeze interval	Does not represent total lifetime freeze time
JANUS_FRZ_DEP	1	Enables dependency-related freeze handling	Runtime configuration

- UNIVERSITY OF INFORMATION TECHNOLOGY -

Variable	Value	Meaning	Note
JANUS_ FRZ_ACT	0	Actuation-related freeze control	Runtime configuration
JANUS_ UNI_K	3.0	Primary univariate diagnostic threshold factor	Supporting detection logic
JANUS_ UNI_K2	2.0	Secondary univariate diagnostic threshold factor	Supporting detection logic
JANUS_ FFT_ KMIN	400	Minimum FFT-related fitting/configuration parameter	FFT fitting
JANUS_ FFT_ TREND_ MAX	3.0	Maximum accepted trend criterion for FFT fitting	FFT fitting gate
JANUS_ CALIB_ TICKS	1200	Number of ticks used for calibration	Offline/runtime calibration configuration
JANUS_ DCALIB_ TICKS	1500	Number of ticks used for decision-calibration gating	Calibration/evaluation
JANUS_ DEC_ DEBUG	1	Enables decision debugging output	Debug configuration
JANUS_ ACT_ WARMUP_ TICKS	5	Number of warm-up ticks before actuation is enabled	Test-only configuration
JANUS_ ACT_URL	http://10.105.196.166:8080/scale	Endpoint used by JanusScale to send scaling requests	Runtime actuator endpoint

The configuration should be distinguished from values obtained during offline evaluation. In particular, the value **58.7343** appears in earlier evaluation outputs, but it should **not** be presented as the current production UCL. Runtime logs from **2026-09-14 through 2026-09-18** report:

```
ucl=default(138.8896)
```

Therefore, this report treats **138.8896** as the UCL value observed in those runtime logs, while **58.7343** remains an offline evaluation value rather than a verified current production setting.

Similarly, JANUS_ACT_WARMUP_TICKS=5 is explicitly associated with the test configuration. It should not be interpreted as the intended permanent production warm-up value.

The configuration parameters above define the runtime operating envelope of the current implementation; they should not by themselves be interpreted as demonstrating that any particular parameter value is optimal.

5.10 Kubernetes / Actuator Integration

The **Kubernetes / Actuator Integration** provides the enforcement interface between the JanusScale decision pipeline and the Kubernetes workload. The inspected actuator is hosted at 10.105.196.166 and exposes a FastAPI `/scale` endpoint for receiving scaling requests.

Requests to the actuator are authenticated using an **HMAC-SHA256 signature** supplied through the `x-signature` HTTP header. The actuator validates the request before proceeding with the scaling operation. This provides request-level authentication and integrity checking at the application layer.

The actuator also applies a deployment whitelist. The inspected configuration includes the `bank-of-anthos/frontend` deployment as an allowed target. Therefore, the actuator does not expose the scaling operation as an unrestricted interface to arbitrary Kubernetes deployments.

After validation, the actuator reads the current replica state and applies the requested replica count through the Kubernetes **Deployment scale subresource**. A per-deployment `asyncio` lock is used to serialize concurrent scaling operations for the same deployment.

The `expected_cur` value supplied by the decision pipeline is retained as **audit/context information**. It does not independently prevent the actuator from applying the requested target when the observed current replica count differs from that contextual value.

Requests whose desired replica count is already equal to the current replica count are handled as **no-ops**, avoiding an unnecessary Kubernetes scaling operation.

After a scaling request is applied, the actuator performs **post-change replica-state verification**. The resulting state is included in the actuator's audit information so that the requested transition can be correlated with the observed Kubernetes state.

The actuator also writes JSONL audit records on a **best-effort** basis. Audit logging provides operational traceability but is not itself the enforcement mechanism.

For the observed closed-loop event, JanusScale generated a target of 5 replicas from a current state of 4, and the actuator recorded the successful transition:

```
ACT ok dep=frontend namespace=bank-of-anthos desired=5 applied_from=4
applied_to=5
```

The actuator application itself runs over **plain HTTP**. TLS is not provided by the inspected application layer, so HMAC-SHA256 should be described specifically as application-level request authentication/integrity protection rather than as transport encryption.

Evidence

The implementation evidence for this section should include two source excerpts:

1. **The actuator /scale endpoint**, showing the `x-signature` verification, whitelist check, Kubernetes scale operation, lock, and post-change verification.
2. **A corresponding JSONL audit entry**, demonstrating the recorded replica transition.

These excerpts should be inserted here once the actuator source and audit-log evidence are included in the report.

6. Experiment and Evaluation

6.1 Experimental Setup

The experiments were conducted in a Kubernetes environment running the **Bank of Anthos** workload. JanusScale operated as the monitoring, anomaly detection, attribution, decision, and enforcement pipeline, with a dedicated actuator service responsible for applying replica-count changes to Kubernetes.

The actuator used in the recorded runtime evaluation was hosted at `10.105.196.166`. JanusScale submitted scaling requests to its `/scale` endpoint, while the actuator interacted with the Kubernetes Deployment scale subresource and verified the resulting replica state after a requested change.

The recorded runtime configuration was supplied through `run.sh`. The main parameters relevant to the experiment were:

Table 6.1: Experimental Runtime Parameters (run.sh Values Used in the Evaluation).

Parameter	Value
JANUS_COOL	100
JANUS_BAND	0.2
JANUS_FRZ	1
JANUS_FRZ_REL	8
JANUS_FRZ_MAX	400
JANUS_FRZ_DEP	1
JANUS_FRZ_ACT	0
JANUS_UNI_K	3.0
JANUS_UNI_K2	2.0
JANUS_FFT_KMIN	400
JANUS_FFT_TREND_MAX	3.
JANUS_CALIB_TICKS	1200
JANUS_DCALIB_TICKS	1500
JANUS_DEC_DEBUG	1
JANUS_ACT_WARMUP_TICKS	5
JANUS_ACT_URL	http://10.105.196.166:8080/scale

- UNIVERSITY OF INFORMATION TECHNOLOGY -

JANUS_ACT_WARMUP_TICKS=5 was used for the recorded test configuration and should not be interpreted as the permanent production warm-up setting.

For the anomaly-detection threshold, the runtime logs from **2026-09-14 through 2026-09-18** report:

```
ucl=default(138.8896)
```

Therefore, **138.8896** is the UCL value reported by the recorded runtime logs. The previously evaluated value **58.7343** is not treated as the current production UCL.

The experimental environment can be summarized as:

*Kubernetes Cluster → Bank of Anthos → JanusScale Runtime Pipeline
→ Actuator(10.105.196.166) → Kubernetes Replica State*

The remaining environment characteristics, including the **Kubernetes version, node count, and detailed Bank of Anthos deployment configuration**, should be reported from the corresponding cluster snapshot rather than inferred from JanusScale runtime logs. Until that snapshot is available, these values remain unspecified rather than being estimated.

6.2 Dataset / Runtime Trace

The runtime evaluation uses the telemetry and decision trace recorded on **2026-09-18** in `ing_20260918T042148Z.log`. The relevant event demonstrates the online JanusScale path from multivariate anomaly detection through attribution, replica-state evaluation, and successful actuation for the `frontend` deployment.

The event sequence is:

UNI → D → ATTR → REP → DECIDE → ACT

The detection record reports a multivariate MEWMA/ T^2 event together with the dimension-level diagnostic values:

```
UNI trig=1 n_k3=2 n_k2=2 max=ram z=22.678 mewma=1 t2=4958.377  
D cpu=1.749 ram=4658.511 rps=5.423 err=21.176 asym=51.639 slow=19.940  
nl=6 fftk=0 tax=3 dcc=0
```

The attribution stage identifies **RAM** as the dominant dimension and associates the anomaly with the `frontend` dependency:

```
ATTR dim=ram d=4658.511 thr=9.149 sig=1 rule=rat dep=frontend z=+20.761
```

The corresponding replica-state record shows the observed workload state:

```
REP ... frontend=4
```

- UNIVERSITY OF INFORMATION TECHNOLOGY -

The decision stage then produces a scale-out target from 4 to 5 replicas:

```
DECIDE dep=frontend dim=ram cur=4 desired=5 max=20 capr=16.05 rps=69.0  
run=18 act=scale_out tax=3 dcc=0 ACTUATED
```

The actuator subsequently reports successful application of the requested state:

```
ACT ok dep=frontend namespace=bank-of-anthos desired=5 applied_from=4  
applied_to=5
```

Actuator Runtime Trace

The actuator log provides a broader history of `frontend` replica changes. The JSONL `ts` values have been converted from Unix epoch to **Vietnam local time (UTC+7)**.

Table 6.2: Timeline of Actuator Frontend Replica-Change Events.

Timestamp (UTC+7)	From	To	Source (src)
2026-09-14 10:28:42.976956	5	3	10.105.196.176
2026-09-14 10:36:04.891704	3	1	10.105.196.176
2026-09-14 16:35:26.678533	5	6	10.105.196.176
2026-09-14 17:36:33.002783	6	11	10.105.196.176
2026-09-15 14:19:43.887645	1	3	10.105.196.176
2026-09-15 14:38:57.275398	3	6	10.105.196.176
2026-09-17 15:00:31.121482	1	2	10.105.196.176
2026-09-17 15:56:27.170789	1	2	10.105.196.176
2026-09-17 16:01:36.823728	2	4	10.105.196.176
2026-09-18 18:03:03.654736	4	5	10.105.196.176

The actuator log also contains a rejected request before the applied events:

```
INFO:janus-actuator:{"event":"reject","reason":"bad_signature","src":"127.0.0.1","ts":1789203111.7063944}
INFO: 127.0.0.1:40848 - "POST /scale HTTP/1.1" 401 Unauthorized
```

This provides an additional runtime trace showing that a request with an invalid signature was rejected with HTTP **401 Unauthorized**.

For the `frontend` sequence, the final 4→5 event occurs at **2026-09-18 18:03:03.654736 UTC+7**. This is the actuator event corresponding to the JanusScale closed-loop evidence presented above. The other `frontend` transitions are part of the actuator history, but the supplied actuator log alone does **not** establish that every transition was generated by the same detection/decision mechanism.

6.3 Experimental Scenarios

The recorded experiment started from a stable cluster state in which the runtime `DECDBG` output showed zero dependency counters. To emulate a sudden workload increase, the `frontend` deployment was manually reduced from **5 replicas to 1 replica**. This manual reduction was the experimental fault-injection step and was not a JanusScale scaling decision.

After the reduction, the runtime counter associated with the `frontend` dependency began to increase. The decision state then progressed across successive ticks: the system held its state at **tick 16**, reported **no data at tick 17**, and issued a scale-out action at **tick 18**.

The actuator log provides direct evidence of the resulting replica change:

```
INFO:janus-actuator:{"event":"applied","src":"10.105.196.176","namespace":"bank-of-anthos","deployment":"frontend","from":1,"to":2,"ts":1789632031.1214817}
INFO: 10.105.196.176:56804 - "POST /scale HTTP/1.1" 200 OK
```

Thus, the observed closed-loop sequence was:

*stable state (5 replicas) → manual reduction to 1
→ increasing frontend dependency counter → hold at tick 16 → no data at tick 17
→ scale out at tick 18 → 1 to 2 replicas*

The resulting target of **2 replicas** was lower than the original 5-replica state. The current replica-target calculation is based on the **RPS dimension**, while the multivariate T^2 detector can identify an anomaly from the six-dimensional observation vector. Therefore, if the dominant anomalous contribution in this event was a dimension other than RPS, the detector's anomaly information would not necessarily produce a corresponding increase in the RPS-based replica target.

This provides a **possible explanation** for the observed 1-to-2 scaling result, but the available evidence does not by itself establish that this was the root cause. Confirming it requires the corresponding P3/P4/P5 trace to identify the dimension contributing to the T^2 event and the exact value used by the replica-target calculation.

The experiment therefore demonstrates a complete runtime response from the induced low-replica state to an actuator-applied scale-out, while also exposing a potential mismatch between **multivariate anomaly attribution** and the **RPS-based replica-target calculation**.

6.4 Detection Evaluation

The P3 Detection plane was evaluated using the recorded runtime detection event:

```
UNI trig=1 n_k3=2 n_k2=2 max=ram z=22.678 mewma=1 t2=4958.377
```

This event shows that the detector entered a triggered state ($\text{trig}=1$). Both supporting univariate criteria were also active, with $n_k3=2$ and $n_k2=2$, and **RAM** was identified as the dimension with the largest univariate diagnostic value ($\text{max}=\text{ram}$, $z=22.678$). At the same tick, the MEWMA detector was active ($\text{mewma}=1$) and produced a T^2 statistic of **4958.377**.

The magnitude of the recorded T^2 statistic is substantially above the **runtime UCL of 138.8896** reported in the corresponding September 14–18 runtime logs. Therefore, for this recorded event:

$$T^2 = 4958.377 > UCL = 138.8896$$

The event consequently provides direct runtime evidence that the P3 detector crossed the configured control limit during the experiment.

The detection trace should be interpreted together with the following diagnostic output from the same event:

```
D cpu=1.749 ram=4658.511 rps=5.423 err=21.176 asym=51.639 slow=19.940  
nl=6 fftk=0 tax=3 dcc=0
```

Among the six monitored dimensions, the RAM-related diagnostic value was the largest in this trace. The subsequent attribution record also identified RAM as the attributed dimension:

```
ATTR dim=ram d=4658.511 thr=9.149 sig=1 rule=rat dep=frontend z=+20.761
```

This provides a consistent runtime chain from the P3 detection event to the P4 attribution result.

Detection evidence summary

Table 6.3: Detection Evidence Summary (P3 Detection Record Fields).

Quantity	Recorded value
Detection trigger	1
MEWMA active	1
T^2	4958.377
Runtime UCL	138.8896
Primary univariate dimension	ram
RAM diagnostic	22.678
n_k3	2
n_k2	2

The available evidence supports **one directly documented runtime detection event** in this evaluation trace. The supplied evidence does not establish a larger count of independent P3 detection events, so additional events should not be claimed without corresponding log records.

6.5 Attribution Evaluation

The P4 Attribution plane was evaluated using the attribution event corresponding to the P3 detection described in Section 6.4:

ATTR dim=ram d=4658.511 thr=9.149 sig=1 rule=rat dep=frontend z=+20.761

This record provides four relevant attribution outputs. First, the attributed anomaly dimension is **RAM** (dim=ram). Second, the recorded diagnostic value is **4658.511**, compared with an attribution threshold of **9.149**. Third, the attribution condition is active (sig=1) and was produced using the **rat** rule. Finally, the anomaly is associated with the **frontend dependency** (dep=frontend), with a standardized diagnostic value of **+20.761**.

The attribution result is consistent with the P3 detection trace:

UNI trig=1 n_k3=2 n_k2=2 max=ram z=22.678 mewma=1 t2=4958.377

Thus, the runtime evidence forms the following sequence:

P3 Detection → RAM identified as dominant dimension

→ P4 attributes the event to frontend → P5 receives dep = frontend and dim = ram

The subsequent decision trace confirms that these attribution fields were consumed by the decision stage:

- UNIVERSITY OF INFORMATION TECHNOLOGY -

```
DECIDE dep=frontend dim=ram cur=4 desired=5 max=20 capr=16.05 rps=69.0  
run=18 act=scale_out
```

This provides evidence that P4 did not merely produce a diagnostic log; its attributed dependency and dimension were propagated into the subsequent scaling decision.

Attribution evidence summary

Table 6.4: Attribution Evidence Summary (P4 Attribution Record Fields).

Field	Recorded value
Attributed dimension	ram
Diagnostic value (d)	4658.511
Attribution threshold (thr)	9.149
Attribution signal (sig)	1
Rule	rat
Dependency	frontend
Diagnostic score (z)	+20.761

The currently available runtime evidence contains **one directly documented attribution event** for this evaluation trace. No additional independent ATTR examples are available in the supplied evidence, so additional attribution events should not be reported without the corresponding full-log records.

6.6 Decision / Scaling Evaluation

The P5 Decision plane was evaluated using the following runtime decision record:

```
DECIDE dep=frontend dim=ram cur=4 desired=5 max=20 capr=16.05 rps=69.0  
run=18 act=scale_out tax=3 dcc=0 ACTUATED
```

The decision was made for the `frontend` dependency after P4 attributed the anomaly to the RAM dimension. The current replica count was **4**, while the calculated desired replica count was **5**, subject to a maximum of **20** replicas. The decision was classified as `scale_out` and was successfully actuated.

Interpretation of decision fields

Table 6.5: Interpretation of P5 Decision Record Fields.

Field	Value	Meaning
dep	frontend	Dependency/workload to which the decision applies.
dim	ram	Dimension retained from the dependency-level anomaly state and used by the decision logic.
cur	4	Current live replica count for the dependency.
desired	5	Replica target calculated by the scaling logic.
max	20	Maximum replica limit applied to the calculated target.
capr	16.05	Estimated RPS-per-replica capacity, obtained from the dependency running mean.
rps	69.0	Current deployment-level RPS.
run	18	Dependency anomaly counter: the number of consecutive ticks for which the dependency remained above the univariate threshold used by the dependency latch logic.
act	scale_out	Decision to increase the replica count.
tax	3	Diagnostic taxonomy bitmask indicating that both metric groups were above their effective DTHR thresholds.
dcc	0	DCC router state; zero indicates that neither DCC route/latch output was active for this decision.
ACTUATE D	-	The decision resulted in an actual actuator call rather than a dry-run or failed action.

The `capr` value is particularly important for interpreting the replica calculation. The implementation normalizes deployment RPS by the **live replica count** before updating the dependency running mean. At decision time, `capr` is read directly from that running mean as the estimated RPS-per-replica capacity. The code then calculates the desired replica count as:

```
capr = dep_mu[kk]
des = int(math.ceil(d_rps[wdep2] / capr)) if capr > 0.0 else cur
```

For the recorded values:

$$\left\lceil \frac{69.0}{16.05} \right\rceil = [4.299...] = 5$$

Therefore, the logged `desired=5` is consistent with the verified RPS-based replica-target formula. The source also confirms that `capr` represents the per-replica RPS running mean;

the implementation explicitly changed this calculation to avoid dividing the historical running mean by the current replica count.

The `run=18` value comes from the dependency-specific anomaly counter. Each dependency maintains its own counter, and the counter is incremented when at least one of its six dimensions exceeds `UNI_K`; the corresponding anomalous dimension is stored in `key_run_dim`. The decision loop requires this dependency counter to reach the configured latch before proceeding with actuation.

The `tax=3` value is a bitmask rather than a continuous score. The implementation sets one bit when at least one of the first three dimensions exceeds its effective DTHR and another bit when at least one of the remaining three dimensions exceeds its effective DTHR. Thus, `tax=3` (1 | 2) indicates that **both groups were represented among the dimensions exceeding their effective DTHR thresholds**.

Finally, `dcc=0` is the output of the DCC Router. The router maintains separate states for its two inputs and encodes them as bit 0 and bit 1 of `router.out`; when neither route is latched, the output is zero. Therefore, `dcc=0` means that **no DCC router state was active at this decision point**. The source also explicitly distinguishes this DCC state from the dimension used for dependency actuation.

Overall, this trace demonstrates that the P5 decision converted the observed state **frontend: 4 replicas, RPS=69.0** into a target of **5 replicas** using an estimated per-replica capacity of **16.05 RPS**, while respecting the configured maximum of 20 replicas. The resulting action was passed to P6 and recorded as **ACTUATED**.

6.7 Actuation Verification

The actuation event was verified using both the JanusScale runtime log and the independent actuator log.

The JanusScale runtime trace records:

```
ACT ok dep=frontend namespace=bank-of-anthos desired=5 applied_from=4
applied_to=5
DECIDE dep=frontend dim=ram cur=4 desired=5 max=20 capr=16.05 rps=69.0
run=18 act=scale_out tax=3 dcc=0 ACTUATED
```

The same runtime trace shows that the **ACT ok** record is emitted immediately before the corresponding **DECIDE ... ACTUATED** record, with the decision specifying a transition from **4 to 5 replicas**.

The independent actuator service records:

```
INFO:janus-actuator:{"event":"applied","src":"10.105.196.176","namespace":"bank-of-anthos","deployment":"frontend","from":4,"to":5,"ts":1789729383.6547356}
INFO:10.105.196.176:56804 - "POST /scale HTTP/1.1" 200 OK
```

The actuator log provides the precise event timestamp:

1789729383.6547356 Unix seconds = 2026-09-18 18:03:03.654736 UTC+07.

The two records agree on the actuation identity:

Table 6.6: Field-Level Comparison Between the JanusScale Log and the Actuator Log.

Field	JanusScale log	Actuator log
Namespace	bank-of-anthos	bank-of-anthos
Deployment	frontend	frontend
Previous replicas	4	4
Target replicas	5	5
Action	ACT ok	event=applied
Request result	-	HTTP 200
Timestamp	Not present on line	1789729383.6547356

Therefore, the evidence confirms a **successful frontend scale-out from 4 to 5 replicas**. The actuator log independently records the applied state change, while the JanusScale trace records the corresponding ACT ok result within the same decision sequence.

A strict timestamp-to-timestamp comparison is **not possible from the displayed JanusScale line**, because that line contains no timestamp. The evidence nevertheless provides a direct field-level match (frontend, bank-of-anthos, 4 → 5) and an independently timestamped actuator event.

6.8 Calibration and Candidate Evaluation

The P7 Calibration and Evaluation plane was evaluated separately from the online control loop. The configurations recorded in `eval.json` represent **shadow candidates** evaluated offline and were not used as the production configuration for the runtime scaling event described in Sections 6.3–6.7.

Shadow candidate configurations

The two candidate configurations recorded in `eval.json` are:

Table 6.7: Shadow Candidate Configurations (eval.json).

ID	UCL	Latch	Cool	dthr[1..6]
B	56.5673	6	100	[25.6655, 6.5573, 6.9970, 6.8730, 29.1200, 20.6941]
C	50.0334	8	100	[24.9689, 6.5488, 6.9570, 6.8484, 27.9837, 20.1079]

These candidate values should not be confused with the production runtime UCL. The closed-loop runtime logs reported **UCL = 138.8896**, whereas the offline evaluation scripts subsequently evaluated configurations using UCL values around **58.7** and other grid values.

Evaluation outputs

The six supplied evaluation files were generated by separate Python evaluation scripts:

- `out_ev3.txt` - `j_ev3_20260828.py`
- `out_far.txt` - `j_far_20260828.py`
- `out_fit.txt` - `j_fit_20260828.py`
- `out_latch.txt` - `j_latch_20260828.py`
- `out_op.txt` - `j_op_20260828.py`
- `out_uni2.txt` - `j_uni2_20260828.py`

The available script search also confirms that the evaluation tooling uses recorded log data and, for the event evaluation, a ground-truth file derived from the same evaluation dataset. The `j_monN_sweep.py` script explicitly accepts a runtime log and a ground-truth file in the `out_ev3.txt` format.

Table 6.8: Summary of out_.txt Evaluation Results.*

File	Key metric(s)	Result	Interpretation
<code>out_ev3.txt</code>	p99 dimension threshold; T^2 events	p99 CPU=3.5097; RAM=1,297,427.5544; RPS=513; ERR=-2.4364; ASYM=-1.1361; SLOW=-0.6544	2/2 recorded CPU events were outside the corresponding raw p99 mask
<code>out_far.txt</code>	FAR, UCL, latch, outside/inside events	At FAR≈1%, UCL=58.6537; latch 6/8/10 produced 48/15/8 fires	No tested (FAR, latch) combination achieved A=0 and B>0
<code>out_fit.txt</code>	Period fitting and phase consistency	Fitted period=3636.00 s; 1120/1332 ticks=84.1% within ±150 s	New fitted timing had 15 s inter-block median-offset spread

- UNIVERSITY OF INFORMATION TECHNOLOGY -

out_latch.txt	Latch/cooldown and outside-window events	W=240, UCL=58.7343; no latch produced A=0	Minimum observed A was 2 at latch=16, cool=0
out_op.txt	Operating-window calibration	UCL=58.7343; FAR=1.0017%; 71,358/86,343 ticks=82.6% passed DCALIB gates	Produced candidate DTHR vector for the selected operating environment
out_uni2.txt	T^2 vs univariate detection	UCL=58.7343; FAR(T^2)=1.0017%; K*=2.0748; FAR(UNI)=1.0017%	UNI triggered 2/2 events that triggered T^2

out_ev3.txt: multivariate event evaluation

The evaluation script j_ev3_20260828.py reported the following upper p99 values for the clean-window dimensions:

<pre>cpu=3.5097 ram=1297427.5544 rps=513.0000 err=-2.4364 asym=-1.1361 slow=-0.6544</pre>

Two recorded events were evaluated:

<pre>2026-08-27T06:33:25Z tick=206240 cpu userservice T2=319.9 value=6.3124 2026-08-27T06:38:25Z tick=206340 cpu userservice T2=284.0 value=5.5763</pre>
--

Both were classified **THAT** because the raw value of the attributed dimension exceeded its corresponding p99 threshold. The output therefore reports **2/2 events outside the clean-window p99 mask**.

out_far.txt: false-alarm grid evaluation

The script evaluated a 3-day clean window with **W=240**, **FRZ=1**, and **cool=100**. Across the tested FAR levels and latches, no configuration satisfied the target condition of **A=0 and B>0**.

For example, at approximately 1% FAR:

Table 6.9: False-Alarm Rate (FAR) Grid Evaluation Results (out_far.txt).

FAR	UCL	Latch	nfire	A_ngoai	B_trong	A_moi_ngay
1.00%	58.6537	6	48	9	39	3.00
1.00%	58.6537	8	15	6	9	2.00
1.00%	58.6537	10	8	3	5	1.00

The script explicitly concludes that no point in the tested (FAR, LATCH) grid achieved $A=0$ and $B>0$.

out_fit.txt: temporal fitting evaluation

The temporal fitting evaluation searched a period grid from **3560 to 3700 seconds** and obtained:

- fitted period: **3636.00 s**
- previous model period: **3622.50 s**
- phase: **2820.0 s**
- **1120/1332 ticks (84.1%)** within ± 150 s of the fitted timing reference
- median-offset spread across the three blocks: **15.0 s**
- previous-model spread: **534.3 s**

The script therefore concludes that the new timing reference is suitable for the subsequent j_uni/j_w2 evaluation.

out_latch.txt: latch and cooldown evaluation

This evaluation used **W=240**, **UCL=58.7343**, **FRZ=1**, **rel=8**, **max=400**, over **86,343 ticks (3 days)**.

The predefined criterion was to minimize events outside the 3622-second clean-window reference. No tested latch value achieved $A=0$.

For **cool=100**, for example:

Table 6.10: Latch and Cooldown Evaluation Results (out_latch.txt).

Latch	nfire	nev_total	A_ngoai	B_trong	A_moi_ngay
2	141	140	67	73	22.33
4	92	91	27	64	9.00
6	47	47	9	38	3.00
8	15	15	6	9	2.00
10	8	8	3	5	1.00

- UNIVERSITY OF INFORMATION TECHNOLOGY -

12	6	6	3	3	1.00
16	2	2	2	0	0.67

The script's conclusion is that no tested latch achieved zero outside-window events.

out_op.txt: operating-window calibration

The operating-window calibration used:

```
W=240
FRZ=1
UCL=58.7343
FAR=1.0017%
```

A total of **71,358 / 86,343 ticks (82.6%)** passed the three DCALIB gates.

The resulting dimension thresholds were:

```
DTHR =
24.8372,
6.6410,
11.5795,
7.0127,
26.4767,
21.2709
```

The environment-relative changes were:

```
cpu=-0.5%
ram=+1.4%
rps=+66.4%
err=+2.4%
asym=-5.4%
slow=+5.8%
```

The concentration values were:

```
cpu=17.2%
ram=16.1%
rps=17.5%
err=14.8%
asym=21.5%
slow=12.9%
```

The 40% concentration criterion was reported as satisfied, with `max=21.5%`.

The script consequently produced:

```
JANUS_W=240
JANUS_UCL=58.7343
JANUS_DTHR=24.8372,6.6410,11.5795,7.0127,26.4767,21.2709
```

These values are **calibration output**, not the runtime configuration of the closed-loop experiment.

out_uni2.txt: univariate supporting evaluation

The univariate comparison used:

```
W=240 UCL=58.7343 FAR(T2)=1.0017% K*=2.0748 FAR(UNI)=1.0017%
```

The criterion for a univariate trigger was the existence of a tick within $[\text{trigger}-16, \text{trigger}+16]$ for which the maximum absolute standardized dimension exceeded K^* .

Both recorded T^2 events also triggered the univariate criterion:

```
2026-08-27T06:33:25Z tick=206240 cpu userservice T2=319.9
max|z|=7.7985 UNI=BAT
2026-08-27T06:38:25Z tick=206340 cpu userservice T2=284.0
max|z|=5.8727 UNI=BAT
```

Thus, $2/2$ T^2 events were also detected by the univariate criterion at the same reported alarm rate.

Evaluation status

The combined results show that the calibration pipeline was used to examine **detection events, false-alarm behavior, temporal fitting, latch/cooldown behavior, operating-window thresholds, and univariate supporting diagnostics**. However, these experiments do **not** establish that candidate B or C is optimal, nor do they establish that either candidate was promoted to production.

In particular, the fact that out_far.txt found no tested configuration satisfying $A=0$ and $B>0$ is itself an evaluation result and should not be described as a failure of the implementation without further context. The script explicitly reports that the tested decision grid did not produce a clean configuration under that predefined criterion.

Artifact hashes

```
out_ev3.txt      64d403d88f51de9d30443ce94bbc4d1f
out_far.txt      37a975224c4d21b4c656edda5648507
out_fit.txt      80e97b149f56ede556368edf9d6d5b8a
out_latch.txt    b5a62ae0431bd2ace8b82932e1737751
out_op.txt       b86b5675ff180deb1e9e45998cd2537d
out_uni2.txt     4bdd596c5934921fa1a5e56f157971ec
```

6.9 Limitations of the Evaluation

The evaluation provides evidence that the JanusScale pipeline can perform anomaly detection, attribution, decision-making, and Kubernetes scaling under the tested runtime and replay conditions. However, the available evidence is **not sufficient to claim optimal autoscaling performance**.

First, the evaluation does not provide sufficient ground-truth coverage to establish that the selected detection and scaling decisions are consistently correct across representative workload conditions. The available runtime traces demonstrate specific closed-loop events,

but they do not constitute a comprehensive ground-truth dataset for all anomaly and scaling cases.

Second, the evaluation does not provide a sufficiently controlled baseline comparison to quantify the relative effectiveness of JanusScale against alternative autoscaling mechanisms under identical workload conditions. The existing results therefore should not be interpreted as evidence of superiority over HPA, KEDA, or other autoscaling approaches.

Third, the available ablation evidence is insufficient to isolate the contribution of every major component of the pipeline. In particular, the existing candidate and calibration experiments do not establish the independent contribution of FFT preprocessing, MEWMA/ T^2 detection, attribution, and policy-controlled actuation to end-to-end autoscaling outcomes.

A further limitation is the distinction between **candidate/shadow evaluation and production runtime configuration**. The offline evaluation reports include candidate configurations such as UCL values around 50–60, whereas the documented production runtime trace uses a different UCL (138.8896). These results must therefore be analyzed separately. Candidate configurations and shadow results demonstrate what was evaluated offline; they do not establish that those configurations were deployed or used for the production actuation event.

Accordingly, the evaluation should be interpreted as evidence of **implementation functionality and behavior under the tested conditions**, rather than as a proof of globally optimal autoscaling. Future evaluation should expand ground-truth coverage, introduce controlled baseline comparisons, and perform systematic ablation studies while maintaining a strict separation between candidate/shadow configurations and production runtime behavior.

7. Results and Discussion

7.1 Detection Results

The runtime detection result in Section 6.4 provides direct evidence that the P3 detector identified an anomalous condition. The recorded event was:

`UNI trig=1 n_k3=2 n_k2=2 max=ram z=22.678 mewma=1 t2=4958.377`

For the corresponding runtime log, the configured UCL was **138.8896**. The observed statistic was therefore substantially above the threshold:

$$T^2 = 4958.377 > UCL = 138.8896$$

The same event reported `mewma=1`, indicating that the MEWMA detector was active, while the univariate diagnostics identified **RAM** as the maximum contributing dimension (`max=ram, z=22.678`). The subsequent attribution record in Section 6.5 independently identified RAM as the attributed dimension, providing continuity between detection and attribution.

An additional offline result is available from `out_uni2.txt`. Using $K^* = 2.0748$, the univariate detector flagged 2/2 events that were also flagged by the T^2 detector:

Table 7.1: Univariate/Multivariate Detection Agreement Summary (out_uni2.txt).

Metric	Result
UCL used in offline evaluation	58.7343
K^* for UNI	2.0748
T^2 events evaluated	2
Events also flagged by UNI	2/2
UNI detection rate among these T^2 events	100%

The `out_uni2` result therefore shows agreement between the multivariate T^2 and supporting univariate detector for the two evaluated events. It should not be interpreted as a general equivalence between the two detectors because only two events are reported in this evaluation.

Overall, the available evidence demonstrates **runtime anomaly detection and agreement with a supporting univariate diagnostic on the evaluated offline events**, but the sample is too limited to establish general detection performance across workloads.

Additional figures or aggregate detection metrics should be added if further runtime logs or controlled experiments become available.

7.2 Attribution Results

The verified runtime trace provides one direct attribution example in which the detected anomaly was attributed to the **RAM dimension of the frontend dependency**:

```
ATTR dim=ram d=4658.511 thr=9.149 sig=1 rule=rat dep=frontend z=+20.761
```

The attribution result is consistent with the preceding detection event, where RAM was identified as the maximum univariate diagnostic dimension:

```
UNI trig=1 n_k3=2 n_k2=2 max=ram z=22.678 mewma=1 t2=4958.377
```

The main attribution fields are summarized below:

Table 7.2: Attribution Result Field Summary.

Field	Observed value	Interpretation
dim	ram	Attributed anomaly dimension
d	4658.511	Dimension-level diagnostic value
thr	9.149	Attribution threshold
sig	1	Significant attribution
rule	rat	Attribution rule reported by the runtime
dep	frontend	Associated workload dependency
z	+20.761	Standardized attribution value

This result establishes the runtime path **multivariate detection** → **RAM attribution** → **frontend dependency**, after which P5 used `dep=frontend` and `dim=ram` in its scaling decision.

Only this **RAM** → **frontend** attribution event is directly documented in the supplied runtime evidence. No additional verified attribution events for other dimensions or dependencies are currently available. Therefore, the result should be presented as a demonstrated runtime example rather than as evidence that attribution performs consistently across all six dimensions or all workloads.

Additional attribution results can be incorporated if further runtime logs provide ATTR records for other dimensions or dependencies.

7.3 Closed-loop Scaling Evidence

The available runtime evidence documents **one complete closed-loop scale-out event** for the frontend deployment.

The sequence is:

1. **Anomaly detection:** JanusScale registered a high $MEWMA/T^2$ condition:

```
UNI trig=1 n_k3=2 n_k2=2 max=ram z=22.678 mewma=1 t2=4958.377
```

with the corresponding runtime UCL of **138.8896**, so $T^2=4958.377$ exceeded the configured threshold.

2. **Attribution:** The anomaly was attributed to RAM and the frontend dependency:

```
ATTR dim=ram d=4658.511 thr=9.149 sig=1 rule=rat dep=frontend  
z=+20.761
```

3. **Decision:** The Decision layer selected a scale-out from 4 to 5 replicas:

```
DECIDE dep=frontend dim=ram cur=4 desired=5 max=20 capr=16.05  
rps=69.0 run=18 act=scale_out tax=3 dcc=0 ACTUATED
```

4. **JanusScale actuation confirmation:** The JanusScale trace recorded:

```
ACT ok dep=frontend namespace=bank-of-anthos desired=5  
applied_from=4 applied_to=5
```

5. **Independent actuator evidence:** The actuator log independently recorded the corresponding Kubernetes scaling operation:

```
{"event": "applied", "src": "10.105.196.176", "namespace": "bank-of-anthos",  
  "deployment": "frontend", "from": 4, "to": 5, "ts": 1789729383.6547356}
```

The actuator returned HTTP **200 OK**, and the timestamp corresponds to **2026-09-18 18:03:03.654736 UTC+7**.

Thus, the available evidence establishes the following closed-loop chain:

MEWMA/T² Anomaly → *RAM Attribution* → *frontend decision 4→5*
→ *Successful Actuation* → *independent actuator record 4→5*

This is sufficient to document **one real closed-loop scale-out event**.

The preceding frontend scale-down phase is **not established by the available logs**. In particular, the available evidence does not independently prove a manual/instructor transition from 5 to 4 replicas. That event must therefore remain explicitly **unproven** rather than being presented as part of the verified closed-loop sequence.

No additional evidence is required for the statement above. Extending the claim to the preceding scale-down phase would require independent Kubernetes audit/history evidence showing the corresponding 5→4 transition.

7.4 Candidate / Shadow Evaluation

The candidate evaluation provides offline evidence for alternative JanusScale configurations, but these configurations must be distinguished from the production runtime configuration. The evaluated candidates were:

Table 7.3: Candidate/Shadow Evaluation Configurations.

Candidate	UCL	Latch	Cooldown	DTHR
B	56.5673	6	100	25.6655, 6.5573, 6.9970, 6.8730, 29.1200, 20.6941
C	50.0334	8	100	24.9689, 6.5488, 6.9570, 6.8484, 27.9837, 20.1079

These values belong to the **offline/shadow candidate evaluation** and should not be interpreted as production operating parameters. In particular, the documented production runtime trace used **UCL = 138.8896**, rather than the candidate UCL values above.

The associated out_*.txt experiments provide several aspects of the offline evaluation:

- **out_ev3.txt - event evaluation:** two evaluated events were outside the corresponding dimension-wise p99 thresholds (2/2), providing evidence for the tested event-detection criterion.
- **out_far.txt - false-alarm/grid evaluation:** none of the tested FAR/latch combinations achieved the specified condition of $A=0$ and $B>0$. This is an evaluation result on the tested data, not evidence of an implementation failure.
- **out_fit.txt - temporal fitting:** the fitted cycle was **3636.0 s**, with **84.1%** of threshold-crossing ticks within ± 150 s of the fitted phase reference. The median phase differences across the three blocks had a maximum spread of **15.0 s**.
- **out_latch.txt - latch/cooldown evaluation:** no tested latch configuration achieved $A=0$. For `cool=100`, the smallest reported A_{ngoai} was **2** at `latch=16`; across all reported rows, the minimum was **1** for `latch=16`, `cool=0`.
- **out_op.txt - operating-window calibration:** the tested configuration reported **UCL 58.7343**, **FAR 1.0017%**, and **71358/86343 ticks (82.6%)** passing the three-DCALIB criterion. These are offline calibration results.
- **out_uni2.txt - univariate supporting evaluation:** with $K^* = 2.0748$, the UNI detector flagged the same 2/2 events flagged by T^2 .

Taken together, these experiments provide evidence about candidate parameter behavior, temporal fitting, false-alarm characteristics, latch behavior, and supporting univariate detection on the evaluated data. They do **not** establish that candidate B or C is the optimal

configuration, nor that either candidate was responsible for the documented production scale-out.

Therefore, the results should be separated into two categories throughout the report:

Offline/shadow evidence: candidate B/C and the `out_*.txt` calibration and evaluation results.

Production runtime evidence: the documented September runtime trace, including `UCL=138.8896` and the verified `frontend 4→5` closed-loop scale-out.

This separation prevents offline candidate performance from being incorrectly presented as production operating performance.

7.5 Interpretation of Results

The results provide evidence for several specific aspects of the JanusScale pipeline, but their interpretation should remain limited to the tested events, traces, and offline experiments.

First, the runtime trace demonstrates that the detection pipeline can identify a high multivariate anomaly condition and subsequently associate that event with a specific dimension and dependency. In the documented event, the anomaly was associated with **RAM** → **frontend**, followed by a decision to increase the deployment from 4 to 5 replicas. This establishes the observed behavior of the implemented closed loop for that event; it does not establish detection accuracy across a broader workload population.

Second, the independent actuator record confirms that the documented 4→5 decision resulted in an actual scaling operation. This provides evidence that the implemented detection, attribution, decision, and enforcement components can form a complete runtime path. It does not by itself establish that the selected replica target was optimal or that the scaling action necessarily produced the best workload outcome.

Third, the offline candidate and calibration experiments provide information about the behavior of tested parameter configurations. Results such as the absence of a tested $A=0, B>0$ configuration in `out_far.txt`, the latch results, and the temporal-fitting results describe the behavior observed on the evaluated data. They should not be generalized to unseen workloads or interpreted as long-term accuracy measurements.

The agreement between T^2 and UNI on the two events reported in `out_uni2.txt` also provides a limited consistency check between the multivariate detector and its supporting univariate diagnostic. Because only two events are documented, this result is insufficient to characterize their general agreement or relative detection performance.

Finally, the candidate/shadow results and the production runtime result must remain separate. Candidate B and C were evaluated offline, while the verified production event used a runtime UCL of **138.8896**. Therefore, the offline candidate results cannot be used to explain or quantify the production event unless additional evidence establishes that a particular candidate configuration was actually deployed.

Overall, the available evidence supports the narrower conclusion that **JanusScale demonstrated the intended detection-to-actuation pipeline for a documented runtime event, while offline experiments provided limited evidence about candidate parameter behavior and detector consistency on the tested data.** The evidence does not support broader claims about overall autoscaling performance, long-term detection accuracy, optimality, generalization, or sustained production effectiveness.

7.6 Limitations and Threats to Validity

The current evaluation has several limitations that constrain the conclusions that can be drawn from the results.

1. **Lack of independent ground-truth anomaly labels.**

The available runtime traces do not provide an independent, comprehensive ground-truth labeling of anomalous and non-anomalous states. Therefore, the observed detections cannot be converted into a reliable measure of detection accuracy, precision, or recall across the workload.

2. **Lack of a comparison baseline.**

The evaluation does not contain a controlled comparison against a baseline autoscaling mechanism under the same workload and experimental conditions. Consequently, the results cannot establish relative improvement over another autoscaling approach.

3. **Lack of a systematic ablation study.**

The available experiments do not isolate the contribution of each major component of the pipeline. In particular, the evidence is insufficient to determine the independent effect of FFT preprocessing, MEWMA/ T^2 detection, attribution, and policy-controlled actuation on the final scaling behavior.

4. **Limited observation window.**

The documented runtime evidence covers a limited set of logs and events. This is insufficient to characterize long-term behavior across different workload patterns, anomaly types, operating conditions, or repeated scaling cycles.

5. **No verified explicit Recovery branch.**

The available evidence demonstrates scale-out enforcement, including the verified frontend 4→5 event, but does not establish a distinct, independently evaluated Recovery branch. Recovery behavior should therefore not be claimed as experimentally demonstrated.

6. **No verified GitOps fast/slow path.**

The available implementation and runtime evidence establish the actuator-based scaling path, but do not provide sufficient evidence for the proposed GitOps fast/slow enforcement workflow. The fast/slow GitOps path should therefore remain a proposed architectural concept rather than an evaluated runtime capability.

These limitations mean that the results in this chapter should be interpreted as **evidence of specific implemented behaviors and tested experimental conditions**, rather than as evidence of general, long-term, or optimal autoscaling performance.

7.7 Comparison Between Intended Design and Actual Implementation

The original proposal described a broader architecture than the implementation that is directly verified by the available source code and runtime evidence. The comparison below therefore distinguishes **implemented and verified functionality** from **intended design concepts** that are not supported by sufficient implementation evidence.

Table 7.4: Proposal vs. Actual Implementation Status.

Proposed Item	Actual Implementation Status	Evidence
Event-driven Kubernetes autoscaling	Implemented and verified	Runtime chain demonstrates anomaly detection → attribution → decision → actuator → frontend 4→5.
Streaming telemetry processing	Implemented and verified	FastAPI ingestion service receives compressed telemetry, decompresses and parses samples, maintains per-pod state, and periodically aggregates observations.
OTel 1-second metrics	Intended design; not verified	The proposal specifies OTel/1-second telemetry, but the implementation uses cAdvisor, Hubble, and Envoy metrics with a configured 3-second flush interval.
Six-dimensional monitoring vector	Implemented and verified	Runtime observation contains CPU, RAM, RPS, error rate, asymmetry, and slow-request ratio.
MEWMA/ T^2 anomaly detection	Implemented and verified	Runtime event reports $mewma=1$ and $t2=4958.377$; corresponding runtime UCL is 138.8896.
FT-based temporal modeling	Implemented as preprocessing	FFT is applied to CPU, RPS, ASYM, and SLOW to generate expected signals and residuals. It is part of the preprocessing/detection pipeline.
Proactive FFT-based scaling	Intended design; not verified	The implementation uses FFT to construct expected signals/residuals. Did not use FFT forecasts directly trigger proactive scaling decisions.
GARCH-based anomalous mode	Partially implemented / supporting analysis	GARCH/DCC-related state exists in the implementation, but the verified runtime detection path is based on MEWMA/ T^2 . GARCH does not act as a separate production anomalous controlling actuation.

- UNIVERSITY OF INFORMATION TECHNOLOGY -

Proposed Item	Actual Implementation Status	Evidence
Dependency attribution	Implemented and verified	Runtime record <code>ATTR dim=ram ... dep=frontend</code> links the anomaly to RAM and frontend.
Policy-controlled scaling decision	Implemented and verified	P5 evaluates current replicas, capacity, RPS, limits, cooldown/freeze state, and produces a desired replica count.
Policy/safety controls	Implemented and verified	Cooldown, freeze/rebaseline logic, warm-up, actuator failure handling, stale state handling, and locking are present in the implementation.
Policy-guarded GitOps	Intended design; not verified	The enforcement path uses a dedicated actuator HTTP endpoint rather than a demonstrated GitOps commit/reconciliation workflow.
Fast/Slow GitOps path	Intended design; not verified	No sufficient runtime or source evidence establishes separate Fast/Slow GitOps enforcement paths.
Webhook enforcement	Intended design; not verified	The actuator exposes <code>/scale</code> and performs HMAC authentication and target validation; no independently webhook-enforcement workflow is available.
Authenticated actuation	Implemented and verified	The actuator uses HMAC-SHA256 signatures, target whitelisting, per-deployment locking, and post-action verification.
Kubernetes replica enforcement	Implemented and verified	Independent actuator evidence records <code>frontend from=4 → to=5</code> with HTTP 200.
Self-healing / recovery branch	Partially demonstrated; explicit Recovery branch not verified	Scale-out behavior is demonstrated, but the available evidence does not establish an independently evaluated Recovery branch.
Offline calibration and candidate evaluation	Implemented/evaluated	Candidate configurations B/C and the associated <code>out_*.txt</code> experiments provide offline/shadow evaluation evidence.

- UNIVERSITY OF INFORMATION TECHNOLOGY -

Proposed Item	Actual Implementation Status	Evidence
Automatic promotion of candidates to production	Not verified	The evidence distinguishes candidate/shadow configurations from the documented production runtime configuration; no automatic promotion mechanism is established.

The comparison shows that the implemented system retains the core **event-driven detection-to-actuation loop**, including streaming telemetry, six-dimensional monitoring, MEWMA/ T^2 detection, attribution, policy-based decision-making, and authenticated Kubernetes enforcement.

Several elements of the original proposal remain at the **intended-design** level. In particular, the evidence does not establish production use of **OTel 1-second telemetry, proactive FFT-driven scaling, a separate GARCH anomalous mode, Fast/Slow GitOps, or webhook enforcement**. These concepts should therefore remain explicitly labeled as proposal-level design rather than being presented as implemented capabilities.

The verified implementation should consequently be described according to the narrower architecture demonstrated by the source code and runtime traces, while the broader proposal remains useful as the original design target.

8. Conclusion and Future Work

8.1 Summary

JanusScale was implemented as an event-driven Kubernetes autoscaling pipeline that processes runtime telemetry, constructs a six-dimensional observation, applies FFT-based preprocessing and MEWMA/ T^2 anomaly detection, attributes detected conditions to workload dimensions and dependencies, and converts the resulting state into policy-controlled scaling decisions. The implementation also includes a dedicated actuator for authenticated Kubernetes replica changes.

The runtime evidence verifies one complete closed-loop scale-out event. In the documented event, JanusScale detected a high MEWMA/ T^2 condition, attributed the anomaly to **RAM for the frontend dependency**, selected a 4→5 replica scale-out, and recorded successful actuation. The independent actuator log also recorded the corresponding frontend 4→5 change with HTTP 200, providing independent confirmation of the scaling operation.

Offline calibration and candidate experiments additionally provided evidence about detector thresholds, temporal fitting, latch/cooldown behavior, false-alarm characteristics, and supporting univariate detection. These results remain **offline/shadow evaluation results** and are distinct from the production runtime configuration.

The available evidence therefore supports the conclusion that the implemented JanusScale pipeline can execute the demonstrated **detection** → **attribution** → **decision** → **Kubernetes actuation** loop under the tested conditions. It does not establish optimal autoscaling, long-term detection accuracy, general workload performance, or superiority over alternative autoscaling mechanisms.

8.2 Achieved Objectives

The objectives in Section 1.4 are assessed against the implementation and evaluation evidence presented in Chapters 5–7. An objective is marked **Y** only where the available evidence directly supports it. Objectives that are only partially demonstrated or remain at the proposal level are marked accordingly.

Table 8.1: Objectives vs. Supporting Evidence.

Objective	Achieved? (Y/N/Partial)	Evidence
Implement an event-driven Kubernetes autoscaling framework	Y	The verified runtime chain demonstrates anomaly detection, attribution, scaling decision, and Kubernetes actuation for frontend.
Collect and process runtime Kubernetes telemetry	Y	P1 ingestion receives compressed telemetry and processes cAdvisor, Hubble, and Envoy metrics into deployment-level observations.

- UNIVERSITY OF INFORMATION TECHNOLOGY -

Objective	Achieved? (Y/N/Partial)	Evidence
Construct a multivariate monitoring representation	Y	The runtime pipeline uses six dimensions: CPU, RAM, RPS, error rate, asymmetry, and slow-request ratio.
Apply MEWMA/ T^2 for multivariate anomaly detection	Y	Runtime evidence reports mewma=1, $t^2=4958.377$, with the documented production UCL of 138.8896.
Apply temporal preprocessing to selected telemetry dimensions	Y	FFT preprocessing is implemented for CPU, RPS, ASYM, and SLOW, with expected signals and residuals passed to the detection pipeline.
Attribute detected anomalies to a workload dimension and dependency	Y	The verified runtime event reports dim=ram and dep=frontend.
Convert anomaly information into a policy-controlled scaling decision	Y	The Decision layer evaluated frontend, current replicas, RPS, capacity, limits, and runtime state before selecting desired=5.
Enforce the scaling decision on Kubernetes	Y	JanusScale recorded applied_from=4 applied_to=5; the independent actuator log also recorded frontend 4→5 with HTTP 200.
Provide authenticated and constrained actuation	Y	The actuator implements HMAC-SHA256 authentication, deployment whitelisting, per-deployment locking, and post-action verification.
Include runtime safety mechanisms around actuation	Y	Cooldown, freeze/rebaseline, warm-up, actuator-failure handling, stale-state handling, and related policy controls are implemented.
Evaluate detector and policy parameters offline	Y	out_ev3.txt, out_far.txt, out_fit.txt, out_latch.txt, out_op.txt, and out_uni2.txt provide offline evaluation evidence.
Evaluate candidate configurations separately from production	Y	Candidates B and C were evaluated offline/shadow; the documented production runtime uses a different UCL (138.8896).

- UNIVERSITY OF INFORMATION TECHNOLOGY -

Objective	Achieved? (Y/N/Partial)	Evidence
Evaluate candidate configurations separately from production	Y	One complete event is verified: anomaly → RAM/frontend attribution → 4→5 decision → successful actuator request → independent 4→5 actuator record.
Demonstrate long-term or general autoscaling performance	N	The available observation window and number of independently verified events are insufficient for this claim.
Demonstrate optimal autoscaling	N	The evaluation lacks sufficient independent ground truth, controlled baseline comparison, and systematic ablation evidence.
Demonstrate an independently evaluated Recovery branch	Partial	Recovery-related implementation logic exists, but the available evaluation evidence does not establish a distinct, independently evaluated Recovery result.
Demonstrate the proposed Fast/Slow GitOps enforcement path	N	The verified runtime enforcement path uses the dedicated actuator; sufficient evidence for the proposed Fast/Slow GitOps path is unavailable.
Demonstrate the proposed OTel 1-second production telemetry path	N	The implementation uses cAdvisor / Hubble / Envoy telemetry and a configured 3-second flush interval; an OTel 1-second production path is not established.
Demonstrate proactive FFT-driven scaling	N	FFT is verified as preprocessing for expected-signal/residual generation, but direct proactive FFT-based actuation is not established.
Demonstrate a separate production GARCH anomalous mode	N	GARCH/DCC-related processing exists as supporting analysis, but the production detection/actuation event is based on MEWMA/ T^2 .

Overall, the evidence supports the **core implemented detection-to-actuation workflow and its associated safety and evaluation components**. The broader proposal objectives that require long-term performance claims, optimality, or unverified architectural mechanisms remain unestablished and are therefore not counted as achieved.

8.3 Current Limitations

The current limitations are as follows:

1. **Limited ground truth:** The evaluation does not provide sufficient independent ground-truth anomaly labels for comprehensive detection-performance assessment.
2. **No baseline comparison:** A controlled comparison against a baseline autoscaling method is not available, preventing claims of relative improvement.
3. **Limited ablation evidence:** The available experiments do not provide a systematic ablation of the main pipeline components.
4. **Limited observation window:** The available runtime and evaluation data are insufficient to establish long-term stability or generalization across broader workload conditions.
5. **Recovery not independently evaluated:** Although recovery-related implementation logic exists, no independent evaluation result is available to establish the behavior of the Recovery path.
6. **Unverified proposed enforcement paths:** The proposed Fast/Slow GitOps and webhook-based enforcement mechanisms are not established by the available runtime evidence; the verified enforcement path uses the dedicated actuator.

These limitations constrain the conclusions to the **specific implementation behaviors and test conditions supported by the available evidence**, rather than long-term performance, generalization, or optimality claims.

8.4 Future Improvements

Future work should focus on strengthening the empirical evidence and independently verifying the parts of the system that are not yet fully established.

1. **Add baseline and ablation studies.** Compare JanusScale against an appropriate baseline autoscaling approach and perform controlled ablations of major pipeline components to quantify their individual contributions.
2. **Independently verify the Recovery branch.** Construct a controlled recovery scenario and independently verify the detection, decision, actuation, and post-recovery behavior of the implemented Recovery logic.
3. **Verify the GitOps Fast/Slow path if implemented.** If the proposed GitOps enforcement architecture is implemented, verify its actual runtime path, including the distinction between Fast and Slow paths and their interaction with the existing actuator.
4. **Expand the runtime dataset.** Collect a longer and more diverse runtime dataset covering additional workload conditions, anomaly types, replica levels, and repeated scaling events. This would provide a stronger basis for evaluating stability and generalization.
5. **Independently verify the preceding scale-down phase.** Obtain an independent Kubernetes audit/history source to establish the origin and timing of the scale-down preceding the verified 4→5 scale-out event. This would clarify whether that earlier transition was produced by JanusScale, manual intervention, or another control mechanism.

6. **Strengthen end-to-end evaluation evidence.** Where possible, align detector events, decision logs, Kubernetes state changes, and independent audit records using common timestamps or transaction identifiers, reducing ambiguity when reconstructing closed-loop behavior.

Appendices

A. Evidence Checklist

The following checklist records the evidence artifacts used to support the implementation and evaluation claims in this report. Checked items indicate evidence that is available and has been used in the report; unchecked items indicate evidence that remains to be independently collected or verified.

- ☒ Full text of `ing_20260918T042148Z.log`, including the verified runtime detection-to-actuation sequence
- ☒ Frontend-related section of `~/actuator/actuator.log`, including the independently recorded replica transition
- ☒ JanusScale source code covering `dep_upd`, `fft_fit`, `fft_slow`, `actuate`, and `flush_loop`
- ☒ Actuator source code covering the `/scale` endpoint, HMAC verification, deployment whitelist, Kubernetes scale-subresource operation, locking, and post-action verification
- ☒ `run.sh` configuration and the `JANUS_*` environment variables relevant to the reported experiments
- ☒ `eval.json`, including the evaluated candidate configurations
- ☒ `out_ev3.txt`, `out_far.txt`, `out_fit.txt`, `out_latch.txt`, `out_op.txt`, and `out_uni2.txt`, with MD5 values recorded where available
- ☐ Independent Kubernetes audit/history evidence for determining the origin of the replica changes preceding the verified 4→5 event
- ☐ Kubernetes cluster version, node topology, and workload configuration used during the experiments

Where an artifact is not independently available, the corresponding claim in the main report is restricted to the evidence that is currently available.

B. Figures Checklist

The following figures document the architecture and evaluation workflow described in the report.

- ☒ **D1 - System Architecture (Closed Loop):** Used in Sections 3.1 and 4.1 to show the logical planes and the online detection-to-actuation loop.
- ☒ **D2 - Telemetry-to-Decision Data Flow:** Used in Section 4.2 to show the transformation from runtime telemetry through preprocessing, detection, attribution, and decision-making.
- ☒ **D3 - Decision and Recovery Flow:** Used in Section 4.3 to represent the decision and safety-control flow. The Recovery branch is treated as an implementation/evaluation item whose independent runtime verification remains limited.
- ☒ **D4 - Calibration and Evaluation:** Used in Section 4.4 to distinguish offline calibration and candidate/shadow evaluation from the production actuation path.
- ☐ **Optional - Actuator Event Timeline:** May be included if the final report requires a visual representation of the independently recorded replica transitions. Any such figure must be derived from the underlying actuator log data.

☐ **Optional - MEWMA/ T^2 Time-Series Chart:** May be included if the required time-series data can be extracted from the available logs or monitoring data. The figure should not imply a continuous time series if only individual runtime events are available.

C. Tables Checklist

The following tables provide the main structured evidence and traceability information used throughout the report.

- ☒ **Functional Requirements Table - Section 3.2**
- ☒ **Data-Flow-by-Plane Mapping Table - Section 4.5**
- ☒ **Runtime Configuration Parameters Table - Section 5.9**
- ☒ **Actuator Event Timeline Table - Section 6.2**
- ☒ **Shadow Candidate and DTHR Summary - Section 6.8**
- ☒ **out_*.txt Evaluation Results Summary - Section 6.8**
- ☒ **Proposal vs. Actual Implementation Comparison - Section 7.7**
- ☒ **Objectives vs. Supporting Evidence - Section 8.2**

D. Claims That Must Not Be Overstated

The following restrictions apply when interpreting the implementation and experimental results presented in this report.

1. **Replica-change provenance must not be assumed.**
The report must not state that the teacher manually scaled `frontend` from 5 to 4 and that JanusScale restored it to 5. The available evidence establishes a successful JanusScale actuation from 4 to 5, but does not independently establish the origin of the preceding 5→4 transition.
2. **Optimal autoscaling must not be claimed.**
The available evidence does not provide sufficient independent ground truth, baseline comparison, and systematic ablation to establish that JanusScale performs optimal autoscaling.
3. **The MEWMA window must not be described as 240 seconds.**
`w=240` represents 240 observations. The runtime `flush_loop` operates at an approximately 3-second configured interval; therefore, the window parameter should not be equated directly with a 240-second duration.
4. **The offline UCL must not be presented as the current production UCL.**
The value `58.7343` belongs to the offline evaluation artifacts. Runtime logs from the reported period use `ucl=default(138.8896)`.
5. **GitOps Fast/Slow enforcement must not be presented as implemented without corresponding evidence.**
The proposed GitOps architecture remains distinct from the verified dedicated HTTP actuator path unless additional source and runtime evidence establishes the GitOps implementation.
6. **Recovery must not be described as a separate P9 plane.**
Recovery is treated as a branch or control behavior associated with the existing architecture. Its independent runtime evaluation is not sufficiently established to justify introducing a separate P9 plane.

7. Candidate/shadow configurations must not be presented as production configuration.
Candidates B and C and the associated out_*.txt evaluation artifacts belong to offline/shadow evaluation. They must not be described as the currently running production configuration without evidence of promotion.
8. **TLS must not be claimed at the actuator application layer.**
The verified actuator endpoint operates over plain HTTP. HMAC authentication provides request authentication but does not establish transport-layer TLS.
9. **Proposal-level mechanisms must remain clearly distinguished from verified implementation.**
OTel 1-second metrics, proactive FFT-based scaling, a separate GARCH anomalous mode, a fully policy-guarded GitOps architecture, and complete webhook enforcement must not be presented as fully implemented unless corresponding source and runtime evidence is available.
10. **Offline evaluation results must not be generalized beyond the tested data.**
Results such as the two detected events in out_uni2.txt, the tested false-alarm configurations, or the fitted periodicity results describe the evaluated datasets and configurations only. They do not by themselves establish general long-term performance.

E. Missing Evidence / Open Questions

- What produced the sequence of replica changes preceding the 4→5 event (5→3, 3→1, 5→6, 6→11, 1→3, 3→6, 1→2, 1→2, 2→4)? Additional logs/audit trail are needed.
- What is the current production runtime configuration if not UCL 58.7343 or the fixed ucl=default(138.8896) - needs confirmation of the configuration actually running at report time.
- Were candidates B/C from eval.json ever promoted to production? No evidence currently supports this.
- Does the Recovery branch in D3 have concrete executable logic in the source, or is it only a conceptual representation on the diagram?
- How exactly are the GARCH/DCC features in flush_loop computed and used in the P5 decision (source excerpt needed)?
- Is there any evidence for a Fast/Slow GitOps path, or is this entirely an unimplemented proposal-level concept?
- What is the scope and reliability of 'best-effort JSONL audit logging' at the actuator - under what conditions could log entries be lost?

References

Works cited in Section 2.1 (Kubernetes autoscaling), 2.3 (MEWMA), 2.4 (FFT), and 2.5 (GARCH/DCC), the primary Kubernetes autoscaling references, and the observability/testing tooling used in the JanusScale implementation.

- [1] Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327. [https://doi.org/10.1016/0304-4076\(86\)90063-1](https://doi.org/10.1016/0304-4076(86)90063-1)
- [2] Lowry, C. A., Woodall, W. H., Champ, C. W., & Rigdon, S. E. (1992). A multivariate exponentially weighted moving average control chart. *Technometrics*, 34(1), 46–53. <https://doi.org/10.2307/1269551>
- [3] Cooley, J. W., & Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90), 297–301.
- [4] Cilium Authors. (n.d.). Cilium Documentation. Retrieved September 2026, from <https://docs.cilium.io/>
- [5] Grafana Labs. (n.d.). Grafana k6 Documentation. Retrieved September 2026, from <https://grafana.com/docs/k6/latest/>
- [6] Engle, R. (2002). Dynamic conditional correlation: A simple class of multivariate generalized autoregressive conditional heteroskedasticity models. *Journal of Business & Economic Statistics*, 20(3), 339–350.
- [7] Grafana Labs. (n.d.). Grafana Documentation. Retrieved September 2026, from <https://grafana.com/docs/grafana/latest/>
- [8] Grafana Labs. (n.d.). Grafana Mimir Documentation. Retrieved September 2026, from <https://grafana.com/docs/mimir/latest/>
- [9] Dang-Quang, N.-M., & Yoo, M. (2021). Deep learning-based autoscaling using bidirectional long short-term memory for Kubernetes. *Applied Sciences*, 11(9), 3835. <https://doi.org/10.3390/app11093835>
- [10] KEDA Authors. (n.d.). KEDA – Kubernetes Event-driven Autoscaling: Documentation. Retrieved September 2026, from <https://keda.sh/docs/>
- [11] Kubernetes Authors. (n.d.). Horizontal Pod Autoscaling. Kubernetes Documentation. Retrieved September 2026, from <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- [12] Liu, B., Guo, J., Li, C., & Luo, Y. (2020). Workload forecasting based elastic resource management in edge cloud. *Computers & Industrial Engineering*, 139, 1–12.
- [13] Yuan, H., & Liao, S. (2024). A time series-based approach to elastic Kubernetes scaling. *Electronics*, 13(2), 285. <https://doi.org/10.3390/electronics13020285>